MDPI

Article

Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

Kun Li [1,*], Ankur Lohachab [1], Michel Dumontier [1] and Visara Urovi [1]

[1] Department of Advanced Computing Sciences, Maastricht University, 6200 MD Maastricht, The Netherlands; ankur.lohachab@maastrichtuniversity.nl (A.L.); michel.dumontier@maastrichtuniversity.nl (M.D.); v.urovi@maastrichtuniversity.nl (V.U.)

* Correspondence: k.li@maastrichtuniversity.nl

Abstract

Blockchain-based consent management makes healthcare data sharing auditable, but public-chain enforcement can reveal the consent policies. Their conditions, such as disease-specific research restrictions, can disclose sensitive information about the data subject. To address this, we propose the Trusted Proof Service (TPS)-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI), in which the TPS generates zero-knowledge proofs of access decisions that a smart contract verifies against committed policies. We compile each policy into a fixed-length program for a reusable zero-knowledge policy interpreter, so all policies share one verification interface and a fixed-format public-chain transcript. We evaluate TPS-ZKPI on Polygon Amoy with supervised and unsupervised attacks by a passive observer inferring hidden policy properties from public-chain metadata. The results show no leakage in any of the 44 supervised tasks under a criterion requiring mean AUC above 0.55 and significance in at least half of 30 splits, and unsupervised clustering recovers no policy structure beyond chance level. Overall, our empirical results show that TPS-ZKPI resists the modeled metadata attacks and provides auditable consent enforcement on a public blockchain.

Keywords: blockchain; consent policy privacy; zero-knowledge proofs; Data Use Ontology (DUO); healthcare data sharing; metadata leakage

1. Introduction

Blockchain technology has emerged as a promising foundation for decentralized and auditable healthcare data sharing [1–3]. Its transparency and immutability allow consent enforcement and authorization decisions to be verified and traced across institutions. However, the same transparency means that consent enforcement on a public blockchain can expose information about the governing consent policies, even when the health data themselves remain encrypted [4,5].

This exposure matters because consent policies in healthcare are often privacy-sensitive [6]. A *consent policy* encodes the conditions under which a data subject authorizes others to access or use their health data, such as who may access the data, for what purpose, and under what constraints. For example, a policy based on biomedical data-use terms may reveal whether a dataset is restricted to disease-specific research, whether ethics approval is required, or whether access is limited to particular institutions or projects. If such a policy is written into a smart contract, embedded in verification

Received:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Blockchains* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

logic, or reflected in public execution traces, unauthorized observers may infer information about the protected resource, the data subject's constraints, or the requester's authorization context.

Existing privacy-preserving blockchain solutions do not address this problem, because they mainly protect data contents, user identities, or transaction details [7–9]. Encrypted data-sharing mechanisms such as attribute-based encryption (ABE) and proxy re-encryption (PRE) control access to data [10,11], and zero-knowledge transaction systems hide asset ownership, values, or participant relationships [12]; none of them protects the confidentiality of the policies that govern access decisions. Hidden-policy ABE [13,14] does hide policy content, but it does not control what a public chain exposes while it enforces the policy. On a public chain, a policy can leak not only through explicit disclosure but also through metadata: the size of the calldata, the layout of the public inputs, the emitted events, the verifier interface, and the gas consumed. Policy-private consent enforcement therefore requires more than data confidentiality and zero-knowledge proofs of correct enforcement. It also requires limiting what the public-chain transcript reveals about the hidden policy.

This paper studies policy-private consent enforcement on public blockchains for healthcare data sharing. The challenge is to let an on-chain verifier check that a requester satisfies a provider-defined consent policy, while ensuring that observers who see only public-chain metadata learn minimal information about the governing policy and the requester's authorization evidence. We focus on Boolean consent policies, including policies derived from biomedical data-use conditions such as terms from the Data Use Ontology (DUO), the Global Alliance for Genomics and Health (GA4GH) standard vocabulary of consent-based data-use conditions [6], and disease-scope constraints [15].

To verify policy satisfaction on-chain without exposing the policy in the public-chain transcript, we propose the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI). In TPS-ZKPI, the data provider commits to a hidden consent policy on-chain and provisions the corresponding policy-bound proving material to a Trusted Proof Service (TPS). A requester submits its policy-relevant authorization evidence, such as attributes, credentials, and access context, to the TPS, which constructs the private witness and generates a zero-knowledge proof that the requester satisfies the committed policy. A smart contract then verifies this proof against the provider's policy commitment. As a result, every authorization decision remains publicly auditable, while the policy logic never appears in the public-chain transcript.

To prevent the transcript itself from revealing the policy, TPS-ZKPI combines four design elements. First, proof generation at the TPS keeps confidentiality-sensitive policy material and requester evidence off the chain. Second, heterogeneous Boolean consent policies are compiled into fixed-length Reverse Polish Notation (RPN) programs, so that every policy has the same length. Third, building on the standard commit-and-prove paradigm, a reusable zero-knowledge policy interpreter evaluates these programs; this avoids policy-specific verifier contracts and allows policies to evolve without changing the public verification interface. Fourth, transcript normalization fixes the format of the calldata, event logs, and public parameters; together with the policy-independent verifier, this leaves the gas consumption independent of the policy content.

We evaluate TPS-ZKPI against a passive public-chain observer who sees only the public-chain transcript. The evaluation expresses consent policy privacy as three objectives: Policy Content Hiding (PCH), Policy Indistinguishability (PI), and Structure and Parameter Privacy (SPP). For these objectives, supervised and unsupervised attacks test whether the public-chain metadata allow the observer to recover hidden policy content, to distinguish otherwise-equivalent policies, or to infer structural and parameter proper-

ties of a policy. The results show no leakage in any of the 44 supervised tasks under our leakage criterion, which flags a task only when its mean area under the receiver operating characteristic curve (AUC) exceeds 0.55 and the attack is statistically significant in at least half of 30 random splits. Unsupervised clustering likewise recovers no policy structure beyond chance level.

This paper makes the following contributions:

- We propose TPS-ZKPI, a TPS-assisted zero-knowledge approach to publicly verifiable and auditable consent enforcement. The soundness of on-chain verification and the auditability of submitted decisions hold without trusting the TPS, whereas policy confidentiality, faithful witness construction, and liveness rely on trusting the TPS.
- We encode each Boolean consent policy as a fixed-length RPN program over a bounded token alphabet and evaluate it in a single zero-knowledge circuit. Combined with policy commitments, this approach provides a policy-independent on-chain verification interface, allowing policies to evolve without changing the verifier.
- We empirically evaluate how much policy information remains in the public-chain metadata on the Polygon Amoy testnet[1] using supervised and unsupervised attacks, and measure its on-chain gas and off-chain proving costs.

The remainder of this paper is organized as follows. Section 2 reviews the technical foundations: cryptographic access control in blockchain healthcare, zero-knowledge proofs, and the Data Use Ontology. Section 3 discusses related work and positions TPS-ZKPI against existing blockchain consent-management, privacy-preserving blockchain, hidden-policy, private policy-evaluation, reusable proving, and metadata-leakage approaches. Section 4 defines the system and trust model, the public-chain adversary, and the privacy objectives. Section 5 presents the TPS-ZKPI design. Section 6 reports the empirical evaluation. Section 7 discusses limitations and future work, and Section 8 concludes.

2. Background

This section reviews the technical foundations relevant to policy-private consent enforcement on blockchains. We first introduce the cryptographic access-control mechanisms commonly used in blockchain-based healthcare data sharing. We then review zero-knowledge proofs and DUO [16], which together provide the basis for the policy representation and verification mechanisms used in our approach.

2.1. Cryptographic Access Control in Blockchain Healthcare

Fine-grained consent management in blockchain-based healthcare data sharing is commonly built on cryptographic access-control primitives, chiefly ciphertext-policy ABE (CP-ABE) and PRE [11,17,18]. In CP-ABE, the encrypting party embeds an attribute-based access policy into the ciphertext, so that only requesters whose credentials satisfy the policy can decrypt the protected data [10]. PRE lets a semi-trusted intermediary transform a ciphertext encrypted for one party into one decryptable by a designated requester, delegating access without exposing the plaintext [18]. Both primitives remain the building blocks of current blockchain healthcare designs; for example, recent schemes combine PRE with on-chain records to delegate access to medical records and Internet of Medical Things (IoMT) data [19,20]. These primitives determine who may obtain the data. However, standard CP-ABE does not keep the access policy confidential because the policy is attached to the ciphertext in plaintext.

[1] https://amoy.polygonscan.com/txs?a=0x89281182b4385AEc4634335c838e5059f4Eca1C6

2.2. Zero-Knowledge Proofs

Zero-knowledge proofs are cryptographic protocols that allow a prover to convince a verifier that a statement is true without revealing any information beyond the validity of the statement [21]. Zero-knowledge proof systems support verifiable computation over private inputs, so that complex predicates can be proven without disclosing the underlying data. In blockchain settings, zero-knowledge proofs are widely used to combine public verifiability with privacy. For example, Zerocash and Tornado Cash use these proofs to conceal transaction details or provide transaction anonymity [9,22].

In healthcare data sharing, zero-knowledge proofs can support authentication before access to encrypted medical records is granted. Sharma et al. [23] combine zk-SNARK-based authentication with PRE to support delegated access to electronic health records (EHRs). In this work, a smart contract verifies a proof of knowledge of the patient's authentication secret, while PRE enables the authorized healthcare provider to decrypt the records. Madine et al. [24] combine transparent zero-knowledge proofs with anonymous credentials to authenticate patients on public and private blockchains while limiting identity disclosure and linkability across interactions. These applications illustrate how zero-knowledge proofs can protect authentication information within a data-sharing workflow.

2.3. The Data Use Ontology

DUO provides a standardized vocabulary for expressing biomedical data-sharing consent conditions, and its terms are widely adopted in genomics and biobanking to enable semantic interoperability [6,16]. A DUO-based policy combines permissions and modifiers, such as general research use, disease-specific research, or a requirement that ethics approval be obtained, into the condition under which access is granted.

DUO Term Categories

Although DUO offers a set of permissions and modifiers, its terms are heterogeneous in how they must be evaluated for compliance. To structure their handling in a zero-knowledge setting, we group DUO terms into four categories:

- **Boolean-only:** simple yes/no conditions that can be represented directly without external validation, e.g. "no restriction" (NRES), general research use (GRU), or "health or medical or biomedical research" (HMB).
- **Ontology-linked:** terms whose evaluation involves consistency checks against external vocabularies or classifications, e.g. disease-specific research (DS), population origins or ancestry research only (POA), or clinical care use (CC).
- **Obligation-required:** terms that impose procedural or temporal obligations, e.g. "publication required" (PUB), "publication moratorium" (MOR), "time limit on use" (TS), or "return to database or resource" (RTN).
- **Credential-required:** terms whose evaluation relies on attested proof of requester status or approvals, e.g. "collaboration required" (COL), "ethics approval required" (IRB), "project-specific restriction" (PS), or "institution-specific restriction" (IS).

Table 1 summarizes representative DUO terms under this category. Some terms reduce to simple Boolean flags, while others rely on ontology references, external obligations, or credential attestations.

3. Related Work

Blockchain-based consent management for healthcare data sharing has been studied extensively, with smart contracts used to record, enforce, and audit consent across institutions [1–3]. These works focus on making consent decisions verifiable and auditable, and the privacy mechanisms surveyed in this domain mainly protect health data contents and

Table 1. Representative DUO terms grouped by evaluation category.

Category	Example Terms	Description
Boolean-only	NRES (DUO:0000004), GRU (DUO:0000042), HMB (DUO:0000006)	Direct yes/no conditions, no external validation required.
Ontology-linked	DS (DUO:0000007), POA (DUO:0000011), CC (DUO:0000043)	Evaluated against external ontologies such as the International Classification of Diseases (ICD-10) or the Mondo Disease Ontology (MONDO).
Obligation-required	PUB (DUO:0000019), MOR (DUO:0000024), TS (DUO:0000025), RTN (DUO:0000029)	Impose obligations such as publication duties, time limits, or data return.
Credential-required	COL (DUO:0000020), IRB (DUO:0000021), PS (DUO:0000027), IS (DUO:0000028)	Depend on credentials, approvals, or institutional/project-specific restrictions.

user identities [5]. The confidentiality of the governing consent policy, and what public-chain enforcement artifacts reveal about it, is typically out of scope.

Beyond consent management, related work addresses individual requirements of policy-private consent enforcement (policy confidentiality, metadata-leakage control, verifiable auditability, or a stable verification interface), but none of the approaches reviewed here combines them on a public chain. Encrypted data-sharing mechanisms, such as CP-ABE and PRE [17,18], control access to encrypted data. However, as noted in Section 2.1, standard CP-ABE exposes the policy itself, while PRE-based delegation may reveal authorization patterns that data encryption is not designed to hide. Transaction-privacy systems, including Zerocash-style protocols [22] and Ethereum-oriented systems such as ZETH and AZTEC [7,8], provide public verification over private transaction witnesses, but they mainly target asset ownership, values, or participant relationships rather than compliance with a provider-defined consent policy.

Hidden-policy CP-ABE schemes reduce direct policy exposure by hiding the access structure in the ciphertext [13,14]. Wu et al. [25] instead realize attribute-based access control (ABAC) by encrypting policy and attribute values under an additively homomorphic cryptosystem. In this scheme, a smart contract evaluates access conditions over encrypted values with assistance from each organization's node, which provides partial decryption results and zero-knowledge proofs of their correctness. However, the logical structure of the policy and its attribute names remain visible on the ledger, and per-attribute match results can be recovered from the ciphertexts and partial decryptions recorded with each decision [25]. The scheme also operates on a consortium blockchain, where enforcement artifacts are not globally visible as they are on a public blockchain. Other approaches provide broader forms of confidentiality. Functional encryption reveals function outputs while hiding inputs [26], but does not by itself provide publicly verifiable authorization evidence. Private-state blockchain systems, including Secret Network and Oasis/Ekiden, protect smart-contract state or execution through encrypted state and trusted execution environments (TEEs) [27,28]. Policy or state confidentiality alone, however, does not ensure that public-chain enforcement artifacts reveal no additional policy information. TPS-ZKPI therefore focuses on limiting policy information exposed during public-chain consent enforcement while preserving public verifiability of each authorization decision.

Self-sovereign identity (SSI) architectures give holders control over their verifiable claims, including the ability to determine which verifiers may access them. Buccafurri et al. [29] support this control through ciphertext-policy attribute-based proxy re-encryption. The holder stores encrypted claims off-chain under a self-defined attribute policy, and a semi-trusted, non-colluding consensus provider re-encrypts them for verifiers whose certified attributes satisfy that policy. A smart contract notarizes hashes of storage, policy-change, and access operations so that they can later be attributed to their parties, and a trusted key-generation authority issues all attribute keys. In healthcare data sharing, Sharma et al. [23] combine zero-knowledge authentication with PRE-based EHR access delegation; the proof establishes knowledge of an authentication secret rather than satisfaction of a hidden consent policy. Zero-knowledge credential presentations likewise let a patient authenticate anonymously on public or private chains [24], and surveys of zero-knowledge proofs for blockchain privacy catalogue this design space [12,30]. TPS-ZKPI focuses on protecting the provider's consent policy during authorization. The requester's claims serve as private witness inputs to a reusable zero-knowledge circuit that verifies satisfaction of the committed policy.

Privacy-preserving access-control and policy-evaluation mechanisms have been studied in credential-based settings built on the eXtensible Access Control Markup Language (XACML) and the Security Assertion Markup Language (SAML) [31], as well as in policy evaluation by multi-party computation (MPC) using homomorphic encryption (HE) and secure function evaluation [32]. These approaches support confidential authorization computation, but they are not primarily designed for settings where policy-evaluation artifacts (calldata, public inputs, emitted events, timing, and gas usage) are permanently and globally visible on a public blockchain. In addition, these approaches do not directly provide compact evidence that a smart contract can verify against a committed, provider-defined policy instance.

Reusable proving systems, including universal-setup succinct non-interactive arguments of knowledge (SNARKs), composable proof systems, and zero-knowledge virtual machines (zkVMs) [33–35], improve maintainability by reducing the need for policy-specific circuits or verifier contracts. However, reusable proving alone does not guarantee policy privacy during public-chain consent enforcement. Policy-dependent program representations, public inputs, calldata layouts, logs, or gas profiles may still reveal information about the hidden policy. This motivates TPS-ZKPI's use of a domain-specific reusable policy interpreter, which preserves proof reuse while exposing only a policy-independent public verification interface.

Studies of metadata leakage on public chains show that cryptographic privacy guarantees may be weakened by observable transaction-level artifacts such as timing and volume patterns [4]. Techniques such as batching, decoys, and traffic shaping can reduce some observable signals [36], but they are usually designed for payment privacy, anonymity sets, or transaction-linkability settings. They do not directly provide hidden-policy compliance verification or auditable consent enforcement. In contrast, TPS-ZKPI treats metadata leakage as part of the policy-privacy objective rather than as a separate transaction-level concern.

Table 2 summarizes the reviewed approaches by policy confidentiality, metadata-leakage mitigation, and verifiable auditability, and notes the scope and limitations of each. Partial indicates limited support or a need for application-specific design; N/A denotes a criterion that does not apply to the cited approach.

Table 2. Positioning of representative approaches against policy-private consent enforcement under public-chain observability.

Approach	Policy confidentiality	Metadata-leakage mitigation	Verifiable auditability	Scope and limitations
Blockchain consent management [1,2]	No	N/A	Yes	Records and enforces consent verifiably on chain, but policy confidentiality and policy-dependent leakage are typically out of scope.
Data / transaction privacy [7,8,17,18,22,37]	No	Partial	Partial	Protects encrypted data access or transaction semantics, but not the governing consent policy or policy-dependent authorization workflow.
Hidden-policy ABE [13,14]	Partial	Partial	Partial	Reduces direct policy exposure, but may leak through workflows, updates, or policy-holder interactions.
Homomorphic hidden-policy ABAC [25]	Partial	N/A	Yes	Hides policy and attribute values on a permissioned ledger, but the logical structure, attribute names, and per-attribute match results remain recoverable from it.
SSI verifier authorization [29]	No	Partial	Partial	Releases the holder's claims only to attribute-qualified verifiers and notarizes access for later attribution, but protects the requester's claims rather than the provider's consent policy.
MPC / HE policy evaluation [32]	Yes	N/A	N/A	Supports confidential policy evaluation off-chain; the cited design does not specify public-chain enforcement or authorization auditing.
Universal SNARKs / zkVMs [33–35]	Partial	Partial	Yes	Supports reusable proving and public verification; policy hiding and a fixed verification interface require application-specific design.
Metadata-leakage defenses [36]	N/A	Yes	Partial	Mitigates transaction timing and volume leakage and supports verifiable execution, but does not directly provide hidden-policy consent enforcement.
TPS-ZKPI	Yes (under TPS trust)	Yes (modeled channels)	Yes	Supports bounded Boolean policies; confidentiality requires trust in the TPS.

4. Problem Formulation

4.1. System and Trust Model

We consider a blockchain-based healthcare data-sharing system with two user roles, following the consent model of Jaiman and Urovi [38]: *data providers*, who provide protected resources and govern access through consent policies, and *data requesters*, who seek access to those resources subject to the applicable policies. TPS-ZKPI mediates this interaction through three components: a TPS operating off-chain, an on-chain access-control manager (ACM), and a data access layer. The provider, which may be the data subject or an organization holding the data [38], defines a consent policy P for a protected resource and registers an on-chain commitment h to the encoded policy. The requester seeks access by submitting policy-relevant authorization evidence, such as attributes, credentials, and access context. The TPS constructs the private witness and generates a zero-knowledge proof of policy compliance. The ACM verifies the proof on-chain and records the authorization outcome, while the data access layer grants access according to the recorded decision.

TPS-ZKPI relies on the following assumptions. First, the provider defines the consent policy and commits to the corresponding policy instance on-chain. Second, the TPS is trusted to preserve the confidentiality of policy-bound proving material and requester-submitted authorization evidence, and to follow the prescribed proof-generation workflow. Third, the commitment scheme is assumed to be hiding and binding, and the zero-

knowledge proof system is assumed to satisfy completeness, soundness, and zero knowledge with respect to the implemented policy-compliance relation. Issuer governance, credential issuance and revocation, and real-world identity verification are treated as external lifecycle functions.

Conceptually, the TPS is an independent proof service, distinct from all data providers; this separation is what makes relocating trust meaningful, since it prevents any data provider from being the single party that holds both the policy and the requester evidence. Possible instantiations of such an independent service include a third-party proof service, a TEE-backed proof service that makes the trust assumption hardware-attested, or MPC/collaborative SNARK proof generation in which no single party sees both policy material and requester evidence. A compromised TPS could substitute inputs if their origin is not independently authenticated, or refuse to generate proofs for legitimate requesters.

The policy commitment and on-chain proof verification ensure that accepted proofs demonstrate satisfaction of the committed policy relation, regardless of how the TPS is instantiated. However, they do not guarantee the provenance of evidence that is not issuer-signed, which remains dependent on the TPS (Table 3). Policy confidentiality, in contrast, is not cryptographically enforced against the TPS itself; it rests on this trust assumption.

Table 3 summarizes, for each actor or failure mode, how each of four security properties (policy confidentiality, authorization correctness, liveness, and decision auditability) holds: whether it is enforced cryptographically, supported only empirically, dependent on the TPS trust assumption, left to external functions, or not provided by the prototype.

Attribute-origin guarantees depend on the evidence type. For DUO terms evaluated from the claims in an issuer-signed profile credential presented by the requester (hereafter, profile-backed terms), the prototype verifies the credential's signature under an issuer key selected by the provider and bound to the policy commitment. The signature scheme, EdDSA-Poseidon, is the Edwards-curve Digital Signature Algorithm (EdDSA) [39] over the Baby Jubjub curve [40], instantiated with Poseidon [41], a hash function designed for efficient evaluation inside zero-knowledge circuits. The credential binds the policy commitment, requester pseudonym, protected resource, claims, validity interval, and credential identifier; the corresponding policy attributes are derived inside the circuit. Thus, subject to signature unforgeability and Poseidon collision resistance, these values cannot be modified without invalidating the proof.

4.2. Public-Chain Adversary

The primary adversary is a passive public-chain observer. The adversary can inspect smart-contract code and monitor public blockchain artifacts across multiple authorization sessions, including policy commitments, submitted proofs, public inputs, emitted events, calldata, and gas usage. The adversary does not observe provider-to-TPS policy material, commitment-opening randomness, requester-submitted authorization evidence, credential witnesses, or the TPS's witness construction process. These data and computations remain off-chain and are not exposed in the public-chain transcript. We assume the off-chain channels themselves are confidential (e.g., TLS-protected).

Each policy-enforcement instance corresponds to an authorization session, in which a requester seeks access to a protected resource under a committed policy. For each such session, the adversary observes a public view

$$view_{\text{pub}} = (h, \pi, o, pp, d_\chi, \tau, id_{\text{req}}, id_{\text{res}}, \nu, meta), \tag{1}$$

where h is the on-chain commitment to the hidden policy, π is the zero-knowledge proof submitted for on-chain verification, o, pp, d_χ, and τ are the remaining public signals of the

Table 3. Security properties of the TPS-ZKPI prototype against each actor or failure mode, and what each rests on: cryptography, empirical evaluation, the TPS following its workflow, or external functions.

Actor or failure mode	Policy confidentiality	Authorization correctness	Liveness	Decision auditability
Passive public-chain observer	$C_{zk,hid}$ for proof and commitment contents; empirical for modeled metadata	$C_{snd,bind}$	N/A	C_{chain}: outcome recorded
Untrusted requester	As for the observer; adaptive-query leakage not evaluated	$C_{snd,ctx}$: cannot forge a proof or alter its bound authorization context	N/A	C_{chain}: pseudonymous request context and outcome recorded on-chain
Workflow-following TPS	✗ against the TPS itself; T against onward disclosure	$C_{snd,bind}$ for the committed relation; T/E for unsigned evidence provenance	T: proves and submits every request	C_{chain} for submitted decisions; ✗ for refusal
Compromised or colluding TPS	✗	$C_{snd,bind,sig}$ for the committed relation and signed claims; E for externally authenticated evidence; ✗ for unauthenticated evidence provenance	✗ if the TPS refuses to prove	C_{chain} for submitted decisions; ✗ for refusal
Forged credential	N/A	$C_{snd,bind,sig}$: signature verified under the policy-bound issuer key	N/A	N/A
Unauthorized or revoked issuer	N/A	E: issuer authorization and revocation status are external	N/A	C_{chain} for the on-chain decision record; E for determining issuer status

C: cryptographic, with subscripts naming the mechanisms it rests on: zk and snd, zero knowledge and soundness of Groth16; hid and bind, hiding and binding of the policy commitment h and the parameter commitment pp; sig, EdDSA-Poseidon signature unforgeability; ctx, on-chain recomputation of the context digest d_χ; chain, integrity of the on-chain record, assuming the underlying chain is secure. T: trust in the TPS following the prescribed workflow; E: external or lifecycle function; ✗: not provided by the prototype; N/A: not applicable.

proof statement (the authorization output, a parameter commitment, a context digest, and a request epoch; Section 5), id_{req}, id_{res}, and ν are the requester, resource, and session-nonce values carried in the access transaction, and *meta* denotes transaction-level metadata exposed by the enforcement layer, such as calldata footprint, event structure, public-input layout, and gas usage. The adversary's goal is to infer hidden information about the committed policy or the authorization interaction from this public view.

In addition to the public-chain observer, we consider the requester as untrusted with respect to policy confidentiality. The requester initiates access requests, observes its own submitted evidence and authorization outcome, and inspects the public-chain transcript. However, the requester does not receive the encoded policy, commitment-opening randomness, or policy-bound proving material, which remain within the provider-to-TPS proof-generation boundary. This restriction matters because a requester that learned the policy could infer sensitive facts about the data subject from the consent terms themselves and, in addition, would learn the policy's full acceptance region (which attribute combi-

nations are granted), enabling targeted evidence construction or probing beyond its own legitimate sessions.

4.3. Privacy Objectives

We operationalize consent policy privacy as three measurable, attack-oriented objectives that cover the information a public-chain observer could extract about a hidden policy: its semantic content, which of several similar policies it is, and its structural form. These correspond to Policy Content Hiding, Policy Indistinguishability, and Structure and Parameter Privacy, respectively.

Policy Content Hiding (PCH). PCH requires that public-chain artifacts should not reveal the semantic content of the hidden consent policy, such as data-use terms, disease categories, research purposes, institutional restrictions, or ethics-related requirements.

Policy Indistinguishability (PI). PI requires that different candidate policies should not become distinguishable from their public verification artifacts when they are compiled into the same fixed public interface. This captures whether two policies can be told apart from commitments, proofs, public parameters, or metadata even when their direct representations are hidden, that is, whether an observer can tell which policy governs an execution.

Structure and Parameter Privacy (SPP). SPP requires that public-chain artifacts should not reveal structural or parametric properties of the hidden policy, such as logical structure, nesting depth, predicate count, or set size.

4.4. Problem Statement

Given P, requester authorization evidence, a globally observable public blockchain, and the TPS trust assumption stated in Section 4.1, the problem is to enforce access to a protected healthcare resource subject to four requirements. First, authorization must be publicly verifiable. The ACM and any observer must be able to verify that the requester satisfies the committed policy without learning the policy or executing it in plaintext. An accepted proof must be bound to the committed policy and the request's authorization context, including the requester pseudonym, resource, and session, to prevent reuse in a different context. Second, the policy must remain confidential. Public-chain artifacts should protect policy content, structure, and parameters and preserve policy indistinguishability as defined by PCH, SPP, and PI (Section 4.3). Requesters should learn no policy information beyond what their own authorization outcomes reveal, and requester evidence should remain off-chain. Third, authorization decisions should be auditable. Each processed request, whether granted or denied, should be recorded on-chain together with its bound authorization context, allowing the decision to be verified later without revealing the policy logic. Fourth, one policy-independent verification interface should serve every policy in the supported class, so that policies can be added or changed without changing the on-chain verifier.

5. The Proposed Approach: TPS-ZKPI

5.1. Overview

As shown in Figure 1, the TPS-ZKPI workflow binds a provider-committed hidden policy to requester-submitted authorization evidence through TPS-side proof generation and on-chain proof verification. The provider registers only a policy commitment on-chain and sends policy-bound proving material to the TPS off-chain (steps 1–2 in the figure). For each access request, the TPS constructs the private witness from the hidden policy material and requester evidence, generates a zero-knowledge proof of the access decision, and submits a fixed-format proof statement for ACM verification (steps 3–4). The ACM

records the authorization outcome in contract storage and emits it in the blockchain event log (as access-granted or access-denied events; step 5); the data access layer consults the recorded decision to serve or refuse the request (steps 6–9).

This separates confidentiality-sensitive proof generation from public verification. Policy material and requester evidence remain off-chain (policy material within the provider–TPS boundary, requester evidence between the requester and the TPS), while the blockchain verifies only a fixed-format proof statement against the provider's policy commitment, alongside session parameters validated by the ACM. In a traditional requester-as-prover design, the requester generates the proof itself and must therefore receive the encoded policy and the commitment-opening randomness, so every requester learns the policy in normal operation. TPS-ZKPI keeps this material within the provider–TPS boundary, so requesters do not learn the policy (Section 4.1).

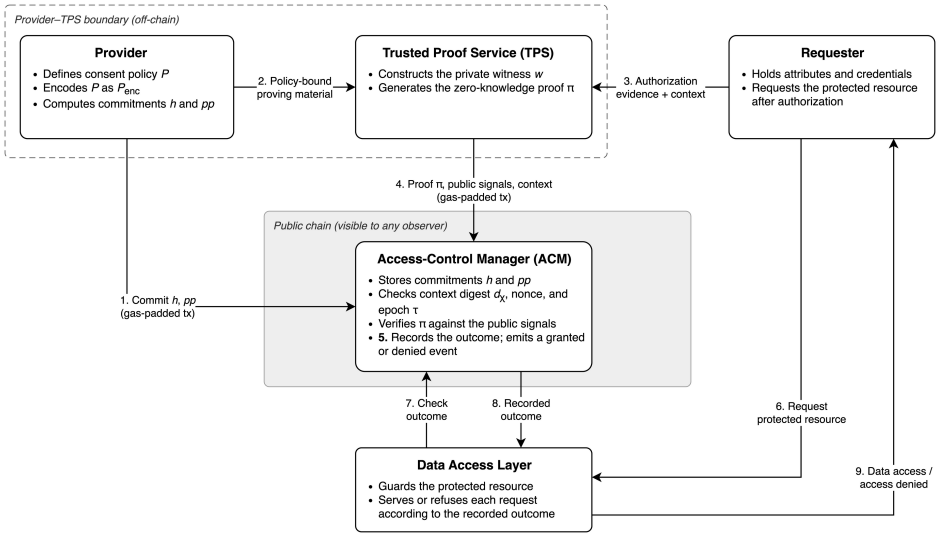


Figure 1. An overview of the TPS-ZKPI workflow.

5.2. Policy Representation and Interpretation

TPS-ZKPI represents heterogeneous Boolean consent policies in a fixed-length, circuit-friendly format. This policy class is chosen for two reasons. First, the supported biomedical consent conditions (DUO-style data-use terms, disease-scope constraints, and requester restrictions) are categorical conditions combined with AND/OR, and conditions that are not natively Boolean enter through fixed external-predicate slots. The generic interpreter also evaluates NOT, but the DUO compiler does not expose negation because its membership witnesses establish inclusion rather than non-membership. Second, a Boolean policy class admits a fixed-length encoding evaluated by a single reusable interpreter circuit, which keeps the verification interface policy-independent, a prerequisite for the privacy objectives evaluated later. Let

$$X = \{0,1\}^N, \qquad B = \{0,1\}^M, \tag{2}$$

denote the requester-attribute assignment space and the external-predicate assignment space, respectively. The assignment space is

$$O = X \times B, \tag{3}$$

where $a \in X$ encodes requester attributes and $b \in B$ encodes predicate outcomes, including credential, ontology-membership, requester-allowlist, and obligation-related checks.

The implemented relation derives these bits from their associated witnesses rather than accepting unconstrained predicate assertions.

Ontology-linked conditions are represented extensionally rather than through in-circuit ontology reasoning. At encoding time, the provider expands an ontology concept into a whitelist of codes and commits the whitelist as a Merkle root among the policy's per-predicate parameters; in-circuit, the predicate check is a set-membership proof of the requested term against this root. A disease-specific policy scoped to diabetes, for example, whitelists the diabetes subtree of the International Classification of Diseases, 10th Revision (ICD-10), whereas one scoped to type-1 diabetes whitelists only the type-1 codes. Expanding the concept hierarchy at encoding time keeps ontology reasoning out of the circuit and preserves the policy-independent interface.

Consent policies are expressed as Boolean formulas over assignment bits:

$$P ::= atom \mid (P \wedge P) \mid (P \vee P) \mid \neg P, \qquad atom ::= x_i \mid y_j. \tag{4}$$

Here x_i ($1 \leq i \leq N$) selects requester-attribute bit a_i and y_j ($1 \leq j \leq M$) selects external-predicate bit b_j. For example, the consent condition "the resource may be used for disease-specific research on diabetes, or for clinical care under approved clinical terms" is expressed as $(x_1 \wedge y_1) \vee (x_2 \wedge y_2)$, where x_1 and x_2 flag the disease-specific-research and clinical-care purposes among the requester attributes, y_1 attests that the requested disease code lies in the committed diabetes whitelist, and y_2 attests that the requested clinical term lies in the approved clinical whitelist. Each policy is compiled into an RPN program over operand tokens and Boolean operator tokens. RPN linearizes arbitrarily nested formulas into a parenthesis-free token sequence that a stack machine evaluates in a single left-to-right pass with a fixed step count, a control structure that maps directly onto a fixed-size circuit and lets one interpreter serve all policies. To support reusable verification, each program is padded to a fixed token length L:

$$P_{\text{enc}} = [t_1, \ldots, t_L], \tag{5}$$

where $t_1, \ldots, t_\ell$ are operand tokens (x_i or y_j) or operator tokens ($\wedge$, $\vee$, $\neg$), together with an explicit program length $\ell \leq L$ carried as a private witness. The interpreter evaluates only the first ℓ tokens; padding positions beyond ℓ are inactive and leave the output unchanged. The example policy above compiles to the token sequence $[x_1, y_1, \wedge, x_2, y_2, \wedge, \vee]$ with $\ell = 7$; the remaining $L - 7$ positions are padding. The implemented circuit uses $N = 13$, $M = 11$, and $L = 64$. We denote the set of length-L programs by Prog_L.

The reusable policy interpreter is a zero-knowledge circuit

$$C : \mathsf{Prog}_L \times \{0, \ldots, L\} \times O \to \{0, 1\}. \tag{6}$$

Given an encoded policy, its program length, and operand values, the interpreter evaluates the RPN program using deterministic stack semantics. The circuit constraints enforce program well-formedness, so no satisfying witness exists for a program that violates the stack discipline. For a well-formed program, the circuit outputs 1 if the requester's evidence satisfies the encoded policy and 0 otherwise. The acceptance relation requires that C output 1.

A new or changed policy is therefore only a different program for the same circuit, so policies can evolve without changing the on-chain verifier.

Scope of the supported policies. Table 4 summarizes how the prototype supports each consent feature (in-circuit, on-chain, committed only, or not provided) and which combinations are supported. In particular, the ACM enforces time-limited use (TS) using public

state. The commit transaction emits the duration, and the first grant emits the requester's authorization window, so policies containing TS do not provide content privacy for that term.

Table 4. Consent-policy features and how far the prototype establishes them.

Consent feature	Enforcement	What is established at authorization time
Unrestricted use (NRES)	In-circuit	Compiled as a standalone unconditional policy; it cannot be combined with another DUO term.
Categorical data-use terms combined with AND/OR	In-circuit	The committed program and its length are evaluated by the interpreter; each term may appear at most once.
Disease, study-design, ancestry, geography, and clinical-term scopes (DS, RS, POA, GS, CC)	In-circuit	Merkle membership of the requested code in a provider-committed whitelist whose root is bound in the parameter commitment.
User-, project-, and institution-specific restrictions (US, PS, IS)	In-circuit	Membership of the holder pseudonym in a provider-committed allowlist for the scope.
Profile-backed terms (GRU, HMB, NMDS, GSO, NPUNCU, NPOA, NPU, NCU; Section 5.4)	In-circuit	EdDSA-Poseidon credential over policy commitment, pseudonym, resource, claims, validity interval, and identifier; the attributes are derived from the signed claims, but subsequent conformity with a declared purpose is not established.
Policy validity window	In-circuit + on-chain	The committed window is checked against the public request epoch, which the ACM requires to equal the current hour.
Time-limited use (TS)	On-chain	The committed duration is also stored and emitted in clear by the ACM; it starts a per-requester window on the first grant, and later requests outside it are denied. TS may appear only conjunctively.
Publication moratorium (MOR)	Committed	The deadline is bound into the policy as an accepted obligation; publication conduct is not observable at request time. MOR may appear only conjunctively.
Prospective consent withdrawal and policy evolution	On-chain	A successor policy supersedes its predecessor, a policy can be revoked, and a resource can be rebound for future decisions; this does not retract data already released or enforce downstream cessation or deletion.
Institutional or ethics approval (IRB)	Not provided	Documentary approval and committee review remain an out-of-circuit authorization step.
Downstream purpose limitation, post-access duties (PUB, COL, RTN), delegation, credential revocation	Not provided	Carried by external governance, the data access agreement, and the issuer lifecycle, outside the proved statement.

"In-circuit" means the condition is part of the proved relation; "on-chain" means the ACM enforces it from public state; "committed" means the value is bound into the policy but compliance is not decided at request time.

5.3. Commitment Binding and Proof Generation

To bind each proof to a provider-defined hidden policy, the provider samples commitment randomness r uniformly from the scalar field of BN254 [42,43], the pairing-friendly elliptic curve over which the proofs are computed, and computes

$$\begin{aligned} q &= \mathsf{H}_{\mathrm{pol}}(P_{\mathrm{enc}}), \\ h &= \mathsf{Com}(P_{\mathrm{enc}}, \ell; r) = \mathsf{Poseidon}(q \parallel \ell \parallel r), \end{aligned} \tag{7}$$

where $\parallel$ denotes concatenation of field-element inputs, q is the Poseidon-based digest of the fixed-length encoded policy, ℓ is the private program length, r is the commitment randomness, and h is the policy commitment stored on-chain[2]. The semantics of an encoded

[2] For example, the commitment $h =$ `0x13bd...e781` of the first policy in the evaluation dataset is recorded in the `PolicyCommitted` event of the Amoy transaction https://amoy.polygonscan.com/tx/0x7b2e8238c0edb0aca858dbd4e1dca0ce0795e1fe3ae03ff6670abec697f60d6a#eventlog.

policy are determined by the pair (P_{enc}, ℓ), since ℓ identifies the active prefix of the padded token array. Accordingly, the circuit supplies the same private value ℓ to both the commitment verifier and the policy interpreter. Assuming Poseidon is collision-resistant, binding the proof to h prevents the prover from changing the committed program length. This prevents the prover from truncating P_{enc} to a shorter prefix that satisfies the stack constraints but skips part of the policy evaluation. This argument relies on the fixed-length encoding of P_{enc}, which keeps the sponge input unambiguous. Hiding is an explicit commitment assumption (Section 4.1) that relies on the high-entropy randomness r.

For an access request, let

$$\chi = (id_{\text{req}}, id_{\text{res}}, \nu), \qquad d_\chi = \mathsf{Poseidon}(id_{\text{req}} \parallel id_{\text{res}} \parallel \nu), \tag{8}$$

where ν is a fresh session nonce. Here id_{req} is the requester pseudonym, which the TPS derives for every request as $\mathsf{Poseidon}(s_{\text{holder}})$ from a private holder secret s_{holder}, so its encoding does not depend on the policy; the circuit enforces this derivation for policies that activate a holder-bound credential or requester-allowlist predicate. The TPS constructs the private circuit witness

$$w = (P_{\text{enc}}, \ell, a, b, r, \omega, \chi), \tag{9}$$

where P_{enc} is the encoded policy, ℓ is its program length, a denotes requester attributes, b denotes external predicate outcomes, each constrained to the result of its check over ω, r is the commitment randomness, and ω denotes auxiliary witnesses for credential- or predicate-dependent checks. The context tuple χ consists of private circuit inputs, while its digest d_χ is public. The public-signal vector is

$$u = (o, h, pp, d_\chi, \tau), \tag{10}$$

where o is the authorization output of the circuit, pp is a fixed-format commitment to the per-predicate parameters, blinded by the commitment randomness r, and τ is the request epoch used by the temporal checks. Although the three components of χ are not separate Groth16 public inputs, the ACM receives them as transaction parameters in order to recompute d_χ; they are therefore visible in transaction calldata.

For the compiled circuit, the Groth16 [21] setup produces the common reference string as a proving key and a verification key, once for the shared circuit:

$$(pk, vk) \leftarrow \mathsf{Setup}(C). \tag{11}$$

The TPS generates a proof

$$\pi \leftarrow \mathsf{Prove}(pk, u, w). \tag{12}$$

An accepting proof ($o = 1$) certifies

$$\begin{gathered} C(P_{\text{enc}}, \ell, (a, b)) = 1, \qquad h = \mathsf{Com}(P_{\text{enc}}, \ell; r), \\ d_\chi = \mathsf{Poseidon}(id_{\text{req}} \parallel id_{\text{res}} \parallel \nu), \end{gathered} \tag{13}$$

and that the selected external-predicate witnesses satisfy their policy-bound verification relations. Before verifying π, the ACM checks that the resource is governed by the stated policy, that τ is current, and that the nonce has not been used for the same requester–resource context. It also recomputes the digest from the transaction parameters and checks that it matches the public signal d_χ. These checks bind the proof to the requester pseudonym, resource, and nonce within a single ACM deployment, preventing its acceptance under a different combination of these values. The ACM then verifies π against vk

and records the outcome of each request. A request is granted only if the proof is valid and $o = 1$; otherwise, it is recorded as a denial.

5.4. Credential-Origin Binding

The circuit reserves one fixed predicate slot for a credential carrying a fixed set of DUO-relevant claims. Let $c_0, \ldots, c_6$ be the private, issuer-signed claim bits for, respectively, research use, health or medical use, methods development, genetic study, ancestry, commercial use, and non-profit use. Let t_{from} and t_{until} define its validity interval, and let γ be its credential identifier. Let d_{DUO} be the fixed credential schema/domain tag for this credential type. From these signed claims, the circuit derives the DUO terms GRU, HMB, no general methods research (NMDS), genetic studies only (GSO), not-for-profit, non-commercial use only (NPUNCU), population origins or ancestry research prohibited (NPOA), not-for-profit organization use only (NPU), and non-commercial use only (NCU). For issuer public key $A = (A_x, A_y)$, the circuit reconstructs the domain-separated message, denoted as:

$$m_{\text{cred}} = \mathsf{Poseidon}(d_{\text{DUO}} \parallel h \parallel id_{\text{req}} \parallel id_{\text{res}} \parallel c_0 \parallel \cdots \parallel c_6 \parallel t_{\text{from}} \parallel t_{\text{until}} \parallel \gamma). \tag{14}$$

Before proof generation, an issuer holding the corresponding private signing key s_{iss} issues the credential by computing

$$\sigma \leftarrow \mathsf{Sign}_{\text{EdDSA-Poseidon}}(s_{\text{iss}}, m_{\text{cred}}). \tag{15}$$

The resulting credential is supplied to the TPS as private off-chain evidence for proof generation. At proof generation, the TPS supplies A, σ, and the credential fields as private witness values. The circuit then enforces

$$\mathsf{Verify}_{\text{EdDSA-Poseidon}}(A, \sigma, m_{\text{cred}}) = 1, \qquad \mathsf{Poseidon}(A_x \parallel A_y) = k_{\text{iss}}, \tag{16}$$

where k_{iss} is the issuer-key commitment contained in the policy-bound parameter commitment pp. The circuit additionally requires $t_{\text{from}} \leq \tau \leq t_{\text{until}}$ and $\gamma \neq 0$. Because this credential is holder-bound, a policy that activates this slot also requires $id_{\text{req}} = \mathsf{Poseidon}(s_{\text{holder}})$, binding the credential to the requester pseudonym in the authorization context. The corresponding DUO attributes are constrained to these claim-derived values.

5.5. Transcript Normalization

Zero-knowledge proofs hide private witness values, but surrounding blockchain artifacts may still leak policy-dependent information to the passive public-chain observer (Section 4). TPS-ZKPI therefore normalizes the public-chain transcript so that the calldata, event logs, and public parameters have the same format for every policy (Table 5).

Table 5. Transcript-normalization mechanisms in TPS-ZKPI.

Artifact	Normalization mechanism
Calldata	Fixed verification interface and fixed-size proof/public-input encoding.
Logs	Fixed event structure with constant topic and payload layout.
Public parameters	Fixed-format handles instead of policy artifacts or credential witnesses.

Even with these fixed formats, gas usage may vary across executions and therefore requires separate analysis as a potential source of policy leakage. Transaction gas consists of a base cost of 21,000, calldata cost, and execution cost. Calldata costs 16 gas per non-zero byte and 4 gas per zero byte [44]. All policies use the same interpreter circuit and verifier contract, keeping the on-chain verification procedure uniform. The authorization outcome o is already disclosed in the event log. Although calldata length is fixed for each

transaction type, its gas cost can still vary with the number of zero bytes in the encoded values.

The proof π is randomized, and the commitments h and pp are blinded using the randomness r. The fields d_χ and id_{req} are Poseidon digests, while ν is a fresh nonce. The epoch τ reflects request timing, and the policy and resource identifiers follow registration order. These fields carry cryptographic representations and public request context rather than plaintext policy content. We evaluate whether the remaining gas variation supports policy inference in Section 6.

6. Experiments and Results

We evaluate PCH, PI, and SPP (Section 4.3) against a passive public-chain observer (Section 6.1). To assess whether this observer can infer hidden policy information from the on-chain observables, we combine supervised attacks with an unsupervised clustering cross-check. The supervised attacks attempt to predict the hidden labels, while the unsupervised analysis groups executions based on their observables without using those labels. We then assess whether the resulting clusters correspond to the hidden labels.

Beyond privacy, we evaluate the system's computational and on-chain costs, including gas consumption for commit and access transactions, calldata sizes, circuit constraint count, and proving time. We also examine how these costs vary with circuit capacity and policy complexity within a fixed circuit, and how offline proof generation throughput scales with workload.

6.1. Adversary and Observables

Under the assumptions stated in Section 4.1, the zero-knowledge proof reveals no information beyond the public statement, and the hiding commitments conceal the committed policy. These cryptographic protections do not, by themselves, prevent policy information from leaking through observable metadata. We therefore evaluate whether the passive public-chain observer defined in Section 4 can recover policy content (PCH, Section 6.4), distinguish between semantically related policies (PI, Section 6.5), or infer policy structure or parameter sizes (SPP, Section 6.6).

To assess whether the observer can achieve these goals, we focus on static metadata from individual policy executions. Each execution consists of a commit transaction followed by an access transaction. We select nine features covering execution cost, calldata length, proof size, and event-log counts and sizes, as these measurements may reveal policy-dependent differences in computation or transaction representation. The features are grouped into four categories.

- **Gas:** `commitGasUsed`, `accessGasUsed`.
- **Calldata length:** `commitCalldataLength`, `accessCalldataLength`.
- **Proof size:** `proofSize`.
- **Event logs:** `commitLogsCount`, `commitLogsSize`, `accessLogsCount`, `accessLogsSize`.

The fixed circuit and commitment-only interface keep calldata length, proof size, and event-log counts and sizes constant across the evaluated policies. We retain these features in the attack inputs, although they provide no variation for distinguishing policies. Within this feature set, only the two gas measurements (`commitGasUsed` and `accessGasUsed`) can vary across executions.

6.2. Dataset and Corpus Generation

The PCH, PI, and SPP evaluations use policies from a dataset of 1,077 distinct policies, run as 1,977 on-chain executions on *Polygon Amoy*, each execution being one commit transaction followed by one access transaction. The policies are produced by a determinis-

tic generator with a fixed seed (`unified-decoupled-2026`), and the number of executions per policy follows from the two execution layers described below. For reproducibility, the generator uses a deterministic test issuer to construct private credential witnesses for policies that require credential verification.

For each policy, the generator selects a set of DUO data-use terms and a logical form that combines them. Five ontology-linked terms (Section 2.3) use code whitelists drawn from external vocabularies: DS uses ICD-10, RS uses the Ontology for Biomedical Investigations (OBI) [45], POA uses the Human Ancestry Ontology (HANCESTRO), GS uses ISO 3166 country codes, and CC uses SNOMED Clinical Terms (SNOMED CT). The other five terms (GRU, HMB, NMDS, GSO, NCU), which we call profile terms, are evaluated using signed claims from the requester's profile credential (Section 5). The generator draws a fixed number of policies from each of six families defined by the terms and connectives they use: a single ontology-linked term, a single profile term, AND-only, OR-only, and mixed AND/OR combinations of several terms, and combinations of profile terms only.

Each policy contains one whitelist for each ontology-linked term it includes. We draw a target whitelist size once per policy from $\{2, 4, 8, 12, 16, 20, 24, 32\}$, independently of its logical form. This target applies to all whitelists in the policy. If a selected scope contains fewer codes than the target, the corresponding whitelist contains only the available codes; this occurs in 8 of the 2,000 whitelists. For analysis, we define a policy's whitelist size as the largest actual whitelist size within that policy. The 147 policies without an ontology-linked term are assigned a whitelist size of 0.

For example, one generated policy combines the DUO terms RS (research-specific use) and DS (disease-specific use). It requires the study design to belong to the OBI *observational and comparative designs* family and the requested record to fall within the ICD-10 *diseases of the circulatory system* chapter. The two conditions are joined by `AND`, so access is granted only when both are satisfied. Each condition is represented by a predicate that checks membership in a whitelist. The RS predicate uses a whitelist of 8 OBI study-design codes, while the DS predicate uses 8 ICD-10 codes drawn from that chapter. Both whitelist sizes fall within the "medium" parameter-size bucket in Table 6.

We execute the policies in two layers. The *coverage layer* executes 1,017 distinct policies once each, spanning the content labels and the categories of policy structure and parameter size and allowing training and testing to use disjoint policies to assess generalization to unseen policies. The *depth layer* executes each of the 60 policies in 30 designed policy pairs 16 times (960 executions), allowing training and testing to use different executions of the same policies to assess whether an observer can distinguish the policies within each pair. For each request, the generator constructs requester evidence that either satisfies or violates the policy, so that the request is granted or denied. In the coverage layer, granted and denied requests are balanced as closely as possible within each group of policies sharing a logical-form family and a set of DUO terms, to reduce associations between authorization outcomes and content labels. Each depth-layer policy receives 8 granted and 8 denied requests. All 1,977 executions produced the planned authorization outcomes (989 granted and 988 denied).

Every execution commits its own instance of its policy with fresh commitment randomness, so no two executions share a policy commitment, a parameter commitment, or an on-chain policy identifier. The executions are submitted in an order shuffled with a fixed seed, independently of the policy labels, so the identifiers assigned in registration order carry no label information. Every content label has at least 33 policies, and Table 6 summarizes the counts for each content label and each category of policy structure or parameter size.

Table 6. Composition of the shared policy corpus

Category	Policies	Executions	Category	Policies	Executions
DUO data-use term (multi-label)			*ICD-10 chapter* (DS policies)		
GRU	302	617	ch0	44	104
DS	422	677	ch1	49	94
RS	412	742	ch2	33	33
POA	392	737	ch3	55	100
GS	382	682	ch4	41	56
CC	392	632	ch5	34	49
NCU	206	506	ch6	48	63
HMB	331	661	ch7	48	48
NMDS	237	387	ch8	33	63
GSO	236	551	ch9	37	67
			Subtotal	422	677
SPP structureType			*SPP nestingDepth*		
trivial	130	235	flat	130	235
and_only	294	579	shallow	210	360
or_only	303	513	medium	335	575
and_or_mixed	350	650	deep	402	807
SPP predicateCombo			*SPP paramType*		
single	257	497	none	147	297
few	645	1185	small	232	457
many	175	295	medium	352	547
			large	346	676
Corpus total	**1077**	**1977**			

ICD-10 labels ch0–ch9 index the ten chapters used: A00–B99, C00–D48, D50–D89, E00–E90, F01–F99, G00–G99, I00–I99, J00–J99, K00–K93, and M00–M99.

The evaluated content labels cover ten DUO data-use terms and ten ICD-10 chapters (Table 6). Evaluating every DUO term and individual ICD-10 code would require a substantially larger on-chain dataset. We therefore evaluate DUO-term presence and ICD-10 chapter membership.

6.3. Attack Methodology

We evaluate each privacy objective primarily through supervised attacks, complemented by an unsupervised clustering cross-check. The *supervised* attacks test whether an observer can learn from labeled examples to predict hidden policy labels from the observables. We measure predictive performance using AUC, with a chance baseline of 0.5. The *unsupervised* analysis tests whether grouping executions by observable similarity recovers hidden policy categories without using labels during clustering. We measure agreement between the clusters and the true labels using the Adjusted Rand Index (ARI) [46] and Normalized Mutual Information (NMI) [47].

6.3.1. Supervised Attacks

For each privacy objective, we define a set of binary classification tasks. PCH tasks predict DUO-term presence or ICD-10 chapter membership (Section 6.4). PI tasks predict which of two policies produced an execution (Section 6.5). SPP tasks distinguish between two categories defined by policy structure or parameter size (Section 6.6).

For each task, we train logistic regression (LR) [48] and random forest (RF) [49] classifiers to evaluate linear and non-linear attacks, respectively. Both classifiers use the nine features listed in Section 6.1. We repeat the evaluation with 30 random seeds. For PCH and SPP, training and testing use disjoint policies. For PI, the attacker knows both policies in a pair, so training and testing use different executions of those same policies (Section 6.5).

For each PCH or SPP task and each seed, we randomly select one execution per policy. We then generate 512 random candidate splits of these selected executions, assigning 70%

of the policies to training and 30% to testing. To preserve class balance, we aim to place 30% of each class in the test set. We discard splits that do not contain both classes in both sets. For each remaining split, we calculate the percentage of each class assigned to the test set and its absolute deviation from the 30% target. We select the split that minimizes the larger of the two class deviations.

We train each classifier on the selected training set and measure its test performance using AUC. The test AUC therefore measures attack performance on policies not seen during training.

For each task, let AUC_s denote the test AUC for seed s, with $S = 30$ seeds. The results report the mean AUC and the AUC gap G_{AUC}, defined as the mean absolute deviation from 0.5:

$$\mathrm{AUC}_{\mathrm{mean}} = \frac{1}{S}\sum_{s=1}^{S}\mathrm{AUC}_s, \qquad G_{\mathrm{AUC}} = \frac{1}{S}\sum_{s=1}^{S}|\mathrm{AUC}_s - 0.5|. \tag{17}$$

Taking absolute deviations before averaging prevents values above and below 0.5 from canceling.

We then assess the statistical significance of the test AUC using a label-permutation test. For each seed and classifier, we keep the test-set prediction scores fixed and randomly permute the test labels $R = 500$ times. We recompute AUC for each permutation to approximate its null distribution under no association between scores and labels. The empirical p-value is

$$p = \frac{k+1}{R+1}, \tag{18}$$

where k is the number of permutations whose AUC is at least as large as the observed AUC. The result is considered significant when $p < 0.05$.

For each task and classifier, we define the significant-seed fraction as the proportion of the 30 seeds for which the label-permutation test yields $p < 0.05$. We report this fraction as %sig.

The PASS/FAIL decision uses the mean AUC and the significant-seed fraction. We use operational thresholds of 0.55 for mean AUC and 50% for the significant-seed fraction. We classify each task as follows:

> **FAIL** if $\mathrm{AUC}_{\mathrm{mean}} > 0.55$ and at least 15 of the 30 seeds yield $p < 0.05$. **PASS** otherwise.

We refer to this rule as the *leakage criterion* and apply it separately to LR and RF. A task is classified as FAIL if either classifier is flagged, and as PASS otherwise. A PASS indicates that neither evaluated attack is flagged under this criterion; it does not establish the absence of policy leakage.

6.3.2. Unsupervised Clustering Attacks

We apply K-Means [50] to the on-chain gas features, setting K to the number of classes ($K = 2$ for policy pairs). We evaluate agreement between the resulting clusters and the true labels using ARI and NMI. Higher values indicate that clustering more closely recovers the hidden categories, suggesting greater privacy leakage. Values close to 0 indicate limited recovery by this attack, but do not establish the absence of leakage. ARI is adjusted for chance [46], whereas NMI is not [47] and can be positive even when the clusters and labels are unrelated (Appendix A); NMI must therefore be compared against a null distribution rather than against 0.

For PCH and SPP, we cluster one execution per policy, the one submitted first (Section 6.2), so that each policy counts once, as in the supervised attacks; this gives 1,077 executions, or 422 for the ICD-10 chapters of the DS policies. For SPP, we perform clus-

tering separately for each dimension defined in Section 6.6, setting K to the number of categories. For both analyses, we report ARI and NMI averaged over 20 K-Means seeds.

To assess the statistical significance of the observed ARI and NMI, we keep the cluster assignments fixed and randomly permute the true labels $R = 200$ times, recomputing both metrics for each permutation. For each metric, we calculate the corresponding p-value using Equation (18), where k is the number of permutations yielding a value at least as large as the observed value. We denote these p-values by p_{ARI} and p_{NMI}. The smallest attainable p-value is $1/201 \approx 0.005$.

For PI, we assess whether the executions of two policies can be distinguished, evaluating each of the 30 designed policy pairs (Section 6.5) separately on its 32 executions, 16 per policy.

For each pair, we apply K-Means with $K = 2$ and compare the resulting clusters with the policy labels. We test the statistical significance of NMI using the label-permutation procedure described above and flag the pair if $p_{\mathrm{NMI}} < 0.05$. We summarize PI using the mean ARI and NMI across pairs and the fraction of flagged pairs, interpreting the latter relative to the nominal 5% significance level.

6.4. Policy Content Hiding (PCH)

This attack predicts DUO-term presence and ICD-10 chapter membership for the ten terms and ten chapters represented in the dataset (Table 6).

For each DUO term, we distinguish policies containing that term from those without it using the supervised attack described in Section 6.3. Table 7 reports the results for all ten terms. Mean AUCs range from 0.494 to 0.535, all below the 0.55 threshold. The significant-seed fraction is highest for POA at 27%, also below the 50% threshold. All ten tasks are therefore classified as PASS under the leakage criterion.

Table 7. Policy Content Hiding: per-term supervised attack over 30 policy-disjoint seeds (one-vs-rest tasks; higher-mean-AUC classifier per task).

Term	AUC (mean $\pm$ std)	AUC gap (mean)	%sig
DUO GRU	0.507 ± 0.034	0.028	7%
DUO DS	0.502 ± 0.022	0.018	3%
DUO RS	0.494 ± 0.029	0.022	3%
DUO POA	0.535 ± 0.024	0.036	27%
DUO GS	0.512 ± 0.023	0.022	0%
DUO CC	0.496 ± 0.027	0.021	0%
DUO NCU	0.506 ± 0.034	0.026	0%
DUO HMB	0.499 ± 0.041	0.031	3%
DUO NMDS	0.523 ± 0.032	0.033	17%
DUO GSO	0.526 ± 0.034	0.033	20%

In each seed, the test set consists of 30% of the 1077 policies (323 policies), each contributing one execution. The positive class of a task consists of the test policies that carry the term; its median size over seeds ranges from 62 (NCU) to 127 (DS).

We apply the same supervised attacks to the ten ICD-10 chapters, restricting the analysis to DS policies, which carry the ICD-10 scope. Each task distinguishes policies associated with a given chapter from those associated with the other chapters. All ten tasks are classified as PASS (Table 8). The tasks for ch0, ch2, and ch6 have mean AUCs of 0.552–0.566, slightly above the 0.55 threshold. However, their significant-seed fractions are at most 10%, below the 50% threshold, so these tasks are classified as PASS.

Table 8. Policy Content Hiding: per-chapter ICD-10 supervised attack over 30 policy-disjoint seeds (higher-mean-AUC classifier per chapter).

Chapter	AUC (mean $\pm$ std)	AUC gap (mean)	%sig
ICD-10 ch0	0.566 ± 0.064	0.081	7%
ICD-10 ch1	0.508 ± 0.070	0.063	3%
ICD-10 ch2	0.561 ± 0.065	0.077	7%
ICD-10 ch3	0.509 ± 0.062	0.050	0%
ICD-10 ch4	0.538 ± 0.085	0.076	10%
ICD-10 ch5	0.517 ± 0.080	0.067	3%
ICD-10 ch6	0.552 ± 0.073	0.068	10%
ICD-10 ch7	0.512 ± 0.071	0.060	7%
ICD-10 ch8	0.539 ± 0.079	0.075	7%
ICD-10 ch9	0.499 ± 0.069	0.057	3%

In each seed, the test set consists of 30% of the 422 DS policies (127 policies), each contributing one execution. The positive class of a chapter consists of the test policies in that chapter; its median size over seeds ranges from 10 to 17.

6.5. Policy Indistinguishability (PI)

PI evaluates whether an observer can identify which of two semantically related policies governs a given execution. We evaluate PI using 30 designed policy pairs, divided into three types with 10 pairs each:

- `semantic_equivalent`: The two policies share the same logical form, consisting of the same DUO terms and Boolean structure, but differ in predicate parameter values, such as whitelist codes.
- `weak_equivalent`: The two policies share the same set of DUO terms but differ in Boolean structure.
- `controlled_difference`: The two policies differ in their sets of DUO terms. For each $k \in \{0, 1, 2, 3, 4\}$, we include two pairs whose policies differ in exactly k of the four SPP dimensions (Section 6.6).

Each policy belongs to one pair, and which of its two policies is P_0 and which is P_1 is drawn at random.

The attacker knows both policies in a pair and can execute each policy to obtain labeled training examples. Given the on-chain observables of an execution, the attacker predicts whether it was governed by P_0 or P_1. For each random seed and policy pair, we split the executions of each policy equally between training and testing, separately for granted and denied outcomes. We train LR and RF on the training executions and evaluate each classifier using test AUC. Each execution commits a separate policy instance (Section 6.2), so training and test executions never share a commitment.

For each pair type and classifier, we first average the test AUCs across the 10 pairs for each seed. To assess significance, we construct a null distribution of this average by repeatedly permuting the test labels within each pair, recomputing each pair's AUC, and averaging across pairs. This yields one mean AUC and one permutation p-value per seed. We then apply the leakage criterion (Section 6.3) using the AUC averaged over the 30 seeds and the fraction of seeds with $p < 0.05$.

All three pair types are classified as PASS (Table 9). The highest mean AUC, 0.549 for `semantic_equivalent` pairs with RF, remains below the 0.55 threshold, with 20% of seeds significant.

6.6. Structure and Parameter Privacy (SPP)

To evaluate SPP, we annotate each policy along four dimensions that capture the structural and parametric properties described in Section 4.3. The four dimensions are

Table 9. Policy Indistinguishability: within-pair supervised attack over 30 seeds, by policy-pair type (higher-mean-AUC classifier per type).

Pair type	Pairs	AUC (mean ± std)	AUC gap (mean)	%sig	med n	Control AUC (%sig)
semantic_equivalent	10	0.549 ± 0.035	0.052	20%	160	0.649 (93%)
weak_equivalent	10	0.520 ± 0.045	0.042	7%	160	0.611 (63%)
controlled_difference	10	0.478 ± 0.026	0.028	0%	160	0.621 (87%)

med n: median test-set size over seeds, summed over the 10 pairs of a type (16 of the 32 executions of each pair). Control: the attack with one added synthetic feature drawn from a standard normal distribution and shifted by 0.75 for P_1; shown for the classifier with the higher control mean AUC (LR for all three types).

logical structure type (`structureType`), predicate count (`predicateCombo`), parameter size (`paramType`), and nesting depth (`nestingDepth`). Each dimension is divided into discrete categories or value ranges. Specifically, `structureType` classifies the logical form joining the DUO terms (Section 6.2) into four categories: `trivial` (a single term with no Boolean connective, e.g. policy logic `"DS"`), `and_only`, `or_only`, and `and_or_mixed`. `predicateCombo` groups policies by predicate-namespace count into `single` (1), `few` (2–3), and `many` ($\geq$ 4); `paramType` groups policies by whitelist size into `none` (no whitelist), `small` (1–4), `medium` (5–16), and `large` ($\geq$ 17); `nestingDepth` groups policies by the maximum nesting depth of the normalized expression into `flat` ($\leq$ 1), `shallow` (2), `medium` (3), and `deep` ($\geq$ 4). In the normalized expression, each term is joined by AND to the predicate that checks it (e.g. `"DS"` becomes `(DS & P[ICD10])`). The policy counts for each category in the dataset are listed in Table 6.

Each pair of categories within a dimension defines a binary classification task. Using the observables, the attacker predicts which of the two categories the executed policy belongs to.

Table 10 reports the results for all pairwise classification tasks. For most tasks, mean AUCs are close to the chance level of 0.5, ranging from 0.490 to 0.546. The `structureType` task `trivial/or_only` and the `nestingDepth` task `flat/medium` both have a mean AUC of 0.559. Although these values exceed the 0.55 threshold, their significant-seed fractions are 27% and 37%, below the 50% threshold, so all 21 tasks are classified as PASS.

6.7. Unsupervised Clustering: A Label-Free Cross-Check

The clustering attacker stated in Section 6.3 tests whether the on-chain observables group by a hidden label without access to that label. Table 11 summarizes the clustering results across the three privacy objectives.

ARI stays within ± 0.006 for every PCH, PI, and SPP target, indicating little agreement between the clusters and the hidden labels, and the PCH and SPP NMI values are at most 0.044. Of the twelve permutation tests for the six PCH and SPP targets, one is nominally significant: the NMI for `nestingDepth` (0.0072, $p_{\text{NMI}} = 0.04$), whose ARI is not ($p_{\text{ARI}} = 0.20$). With twelve tests at the 5% level, one such result is consistent with chance, and all six supervised `nestingDepth` tasks pass (Section 6.6). For PI, 1 of the 30 pairs (3%) has $p_{\text{NMI}} < 0.05$, consistent with the nominal 5% level. These findings complement the supervised results on the same dataset.

6.8. Cost and Performance

At $L = 64$ and eight Merkle levels, the prototype has 81,980 constraints and mean gas costs of 219,757 for commit and 723,565 for access (Table 12). The gas varies only within narrow ranges, from 219,688 to 237,664 for commit and from 723,446 to 741,458 for access. Subtracting the base cost of 21,000 and the calldata cost from each transaction (Section 5.5) leaves an execution cost of 196,964 gas for every commit, 70,400 for every resource registration, and 695,254 and 695,264 for every denied and granted access, respectively, apart

Table 10. Structure and Parameter Privacy: 21 pairwise contrasts across four annotation axes over 30 policy-disjoint seeds (higher-mean-AUC classifier per contrast).

Contrast	AUC (mean ± std)	AUC gap (mean)	%sig	med n
structureType				
trivial / or_only	0.559 ± 0.045	0.063	27%	130
trivial / and_only	0.533 ± 0.050	0.047	7%	127
trivial / and_or_mixed	0.537 ± 0.040	0.042	7%	144
or_only / and_only	0.510 ± 0.038	0.032	7%	179
or_only / and_or_mixed	0.541 ± 0.032	0.045	13%	196
and_only / and_or_mixed	0.506 ± 0.039	0.029	3%	193
predicateCombo				
single / few	0.501 ± 0.037	0.028	7%	271
single / many	0.498 ± 0.038	0.031	3%	130
few / many	0.504 ± 0.029	0.023	3%	246
paramType				
none / small	0.490 ± 0.045	0.038	3%	114
none / medium	0.520 ± 0.044	0.036	10%	150
none / large	0.522 ± 0.044	0.038	13%	148
small / medium	0.532 ± 0.041	0.043	17%	175
small / large	0.518 ± 0.040	0.033	10%	173
medium / large	0.491 ± 0.032	0.026	0%	209
nestingDepth				
flat / shallow	0.533 ± 0.058	0.056	7%	102
flat / medium	0.559 ± 0.056	0.067	37%	140
flat / deep	0.541 ± 0.053	0.055	23%	160
shallow / medium	0.546 ± 0.037	0.049	23%	164
shallow / deep	0.493 ± 0.042	0.034	3%	184
medium / deep	0.507 ± 0.024	0.020	0%	221

med n: median test-set size over seeds.

Table 11. Unsupervised clustering attack (K-Means, K = number of classes, $K = 2$ for pairs) on the on-chain gas features.

Objective	Target	n	#cls.	ARI	p_{ARI}	NMI	p_{NMI}
PCH	DUO terms	1077	10	0.0024	0.10	0.0167	0.69
PCH	ICD-10 chapters	422	10	−0.0008	0.58	0.0437	0.62
PI	policy pairs (16v16)	30	2	−0.0020	—	0.0242	—
SPP	`structureType`	1077	4	−0.0022	0.87	0.0053	0.18
SPP	`predicateCombo`	1077	3	−0.0051	0.69	0.0020	0.65
SPP	`paramType`	1077	4	−0.0004	0.56	0.0023	0.78
SPP	`nestingDepth`	1077	4	0.0022	0.20	0.0072	0.04

For PI, ARI and NMI are means across the 30 designed policy pairs. Each pair is clustered and tested separately, so no single p-value applies to these means (—).

Table 12. Single-point cost profile of the evaluated prototype configuration at $L = 64$. Gas, logs, calldata, and local prover timings are summarized over the 1,977 Polygon Amoy executions.

Metric	Value / source
Circuit size	81,980 constraints; 81,124 wires; $L = 64$
Public interface	4 public inputs plus 1 output
On-chain gas	Commit 219,757; access 723,565 mean gas
Event logs	Commit: 1 log / 180 bytes; access: 2 logs / 296 bytes
Calldata length	Commit 164 bytes; access 580 bytes (constant across policies)
Local witness time	1,140.0 ms mean
Local prove time	3,990.3 ms mean

from 17,940 additional gas on the first transaction of each sending account. Across the 1,977 Amoy executions on the two-vCPU capture server, which ran four capture processes concurrently, mean witness and proving times are 1.140 and 3.990 s.

Table 13. Offline benchmarks of circuit capacity (30 verified proofs per row; every variant keeps $N = 13$, $M = 11$, and five public signals).

Parameter	Capacity	Constraints	Wires	WASM (MB)	zkey (MB)	Witness	Prove	Verify
Program capacity: Merkle levels fixed at 8; mixed policy								
Program length L	32	64,945	64,441	8.6	38.7	285 (343)	411 (562)	11 (16)
Program length L	64	81,740	80,884	8.7	52.2	350 (414)	521 (761)	11 (22)
Program length L	128	134,009	132,449	10.1	87.5	535 (682)	734 (984)	11 (15)
Ontology capacity: $L = 64$; five-slot ontology-heavy policy								
Merkle levels	4	73,876	73,052	8.6	48.3	352 (415)	512 (605)	11 (17)
Merkle levels	8	81,740	80,884	8.7	52.2	347 (354)	499 (539)	11 (16)
Merkle levels	12	89,604	88,716	8.7	56.0	358 (395)	552 (632)	10 (15)
Merkle levels	16	97,468	96,548	8.7	59.9	370 (456)	571 (713)	10 (13)

Timing cells report median (p95) in ms; WASM and zkey are the sizes of the WebAssembly (WASM) witness generator and the proving key, respectively.

To assess how circuit capacity affects proving costs, we generate 30 verified proofs per configuration using rapidsnark on an Apple M1 (Table 13). Increasing program capacity L from 32 to 128 raises the constraint count from 64,945 to 134,009 and median proving time from 0.411 to 0.734 s. At $L = 64$, increasing the number of Merkle levels from 4 to 16 raises the constraint count from 73,876 to 97,468, whereas median proving time rises only from 0.512 to 0.571 s.

Table 14. Policy-content and workload scalability of the evaluated circuit ($L = 64$, eight Merkle levels) on one Apple M1 host using rapidsnark.

(a) Policy content

			Time (ms): median (p95)			
Policy	Tokens	Slots	Witness	Prove	Verify	End-to-end
Simple	3	1	354 (418)	545 (746)	12 (14)	925 (1170)
Ontology-heavy	19	5	349 (391)	506 (550)	10 (12)	880 (939)
Profile-heavy	31	1	337 (349)	492 (522)	10 (11)	1479 (1516)
Max-mixed	61	11	335 (346)	484 (537)	10 (10)	1468 (1538)

(b) Load

Concurrency c	Throughput (proofs/s)	Latency (ms): median (p95)
1	0.805	1230 (1659)
2	0.787	2492 (3805)
4	0.936	4113 (5854)

Tokens: program length ℓ (at most $L = 64$); slots: external-predicate slots the policy uses (of $M = 11$). End-to-end time also includes off-circuit evidence preparation, which for the profile-backed policies includes the harness's per-request issuance of a signed credential (about 0.64 s). Latency is each request's processing time while up to c requests run at once; queueing is excluded. Five initial requests at each concurrency level are excluded from all timings.

Within the fixed circuit ($L = 64$, eight Merkle levels), median witness and proving times vary little across the evaluated policies, ranging from 0.335 to 0.354 s and from 0.484 to 0.545 s, respectively (Table 14). For profile-backed policies, the harness adds about 0.64 s by issuing a fresh credential per request, bringing median end-to-end time to approximately 1.47–1.48 s. Pre-issued credentials can be reused in deployment.

With 100 requests per concurrency level, offline throughput stays between 0.79 and 0.94 proofs/s as concurrency increases from one to four, while median latency rises from 1.2 to 4.1 s (Table 14). These measurements include the harness's evidence preparation and credential issuance; latency excludes queueing.

7. Discussion

Privacy findings. Under the passive public-chain observer model, the evaluated attacks achieve performance close to chance. None of the supervised tasks meets the leakage criterion for FAIL. The highest mean AUC is 0.566 for ICD-10 chapter ch0, with statistically significant results in only 7% of seeds (Section 6.4). The clustering analysis yields one nominally significant result across twelve tests, which is consistent with chance at the 5% significance level (Section 6.7).

The commitment-only interface and fixed circuit avoid exposing policy logic directly in the public transcript, and the gas that follows from them varies only with the transaction type, the authorization output, and the byte cost of values that do not depend on the policy content (Section 6.8).

Costs. Within the evaluated circuit, policy complexity has little effect on on-chain costs. All proofs require the same verification operations and use a fixed public-input format with five signals. The costs of witness generation and proving for larger circuit capacities or different policy contents are incurred off-chain (Tables 13 and 14).

The offline load benchmark in Table 14 rotates the four policy classes through 100 verified requests at each concurrency level. The best observed batch throughput is 0.936 proofs/s at concurrency four, with median and p95 request latencies of 4.113 and 5.854 s; CPU contention makes the concurrency-two result slightly lower. At this throughput, one prover on the benchmark host handles about 80,900 requests per day, or about 53,900 if a third of its capacity is held in reserve; the estimate is conservative, because half of the rotated requests include the harness's credential issuance (Section 6.8). On-chain, each access costs 723,565 gas on average (about 0.022 POL, the native token of Polygon, at the 30 gwei tip of the evaluation; Table 12), and each policy commit adds a one-time 219,757 gas (0.007 POL).

TPS trust and credential provenance. The TPS trust assumption is the main architectural tradeoff. Moving confidentiality-sensitive proof generation to the TPS keeps policy-bound proving material away from requesters and public smart contracts while preserving on-chain verifiability, but it shifts part of the confidentiality burden to the service layer (Section 4). It also affects liveness and accountability. The TPS can refuse to generate a proof, and the on-chain record alone may not reveal the reason for that refusal.

Attribute origin is only partially delegated to the TPS. For the profile-backed DUO terms, a compromised TPS cannot alter the signed claims or the fields bound with them (Sections 4.1 and 5.4). However, because the TPS receives the holder secret and credential witness in the current workflow, holder binding does not protect against a malicious TPS reusing otherwise valid evidence within its signed scope.

Issuer governance and deployment path. The remaining trust anchor is therefore issuer governance. The prototype binds the proof to a policy-selected issuer-key commitment, but it has no independent registry establishing that this key belongs to an accredited healthcare authority, and it does not check credential revocation.

A deployment could add issuer governance without modifying the Boolean interpreter. A governed registry would publish issuer keys together with their associated credential schemas and validity epochs. Each issuer would sign a domain-separated credential containing a holder-key commitment, claim vector, authorization scope, validity interval, and unique credential identifier. The circuit would verify the credential signature and a Merkle proof that the issuer key is active in the registry, while continuing to derive policy attributes from the signed claims. The holder would authorize each request by signing the context digest and request epoch, preventing a TPS that observes one credential presentation from generating proofs for fresh sessions; an epoch-specific sparse-Merkle

or accumulator root would provide a private non-revocation witness for the credential identifier.

The registry and revocation roots can occupy fixed-format public or committed parameter slots, preserving the Boolean interpreter and the shape of the policy-independent public interface, although these checks require a revised circuit and new proving and verification keys. Existing healthcare credentials using Ed25519 or the Elliptic Curve Digital Signature Algorithm require either higher-cost in-circuit verification or an additional SNARK-friendly credential; TPS re-signing cannot establish issuer-origin assurance.

Policy and context binding. The policy commitment binds the active program length ℓ, and the circuit ties the public digest d_χ to the requester pseudonym, resource, and nonce (Section 5.3); an accepting proof therefore can neither skip part of the committed policy nor be accepted under another context. This context-binding guarantee applies within a single ACM deployment. The digest does not bind the transaction submitter, chain identifier, or contract address. Additional checks are therefore needed to prevent another party from copying and submitting the same request first, or reusing the proof on another chain or contract.

Healthcare data-access walkthrough. To complement the synthetic-policy evaluation, we trace an illustrative consent condition through the prototype's compiler, circuit, and ACM, distinguishing cryptographic checks from external responsibilities (Table 15). Consider a dataset whose consent permits diabetes research by researchers affiliated with approved institutions, subject to ethics approval. The provider represents these requirements using the DUO terms DS (restricted to ICD-10 codes E10–E14), IS, and IRB. The compiler encodes DS as a whitelist-membership predicate and IS as membership in an allowlist of requester pseudonyms. The compiler rejects IRB; ethics approval must therefore be reviewed outside the proof before data release. For the compiled DS and IS conditions, an accepted proof establishes that the declared disease code is in the committed whitelist and that the requester pseudonym is in the committed allowlist. Institutional affiliation is attested outside the circuit before the provider adds a requester's pseudonym to the allowlist. Because the allowlist root is bound in the parameter commitment, adding or removing a researcher requires a successor policy. Although IS is supported by the circuit, it is not included in the evaluation corpus (Section 6.2).

The walkthrough also clarifies the responsibilities of deployment participants. The TPS is operated by a third party independent of all data providers (Section 4.1); this could be the operator of a trusted research environment accepted by the participating data providers and requesters. The provider's data access committee, for example, reviews ethics approval outside the proof before data release. For policies that also include profile-backed terms, an institution issues the profile credential, and the provider binds the institution's issuer key into the policy (Section 5.4), subject to the issuer governance discussed above.

Following an on-chain grant, the data access layer checks that the request was granted in the current hour and that the resource remains bound to an active policy. The prototype implements the authorization check, leaving actual data delivery to the deployment. The same check enforces consent withdrawal by rejecting earlier grants once the policy is revoked or the resource is rebound to a successor policy.

Fidelity of the policy encoding. The fidelity of the encoded policy depends on the provider's interpretation of consent and the compiler's expressive capabilities. The provider translates consent text into DUO terms, and the prototype verifies compliance with the resulting policy without validating this translation. Unsupported conditions, such as IRB, are rejected and must be addressed outside the proof. Table 4 lists the supported conditions and restrictions on combining terms. Ontology-linked conditions also

Table 15. The diabetes-research example: what the prototype establishes at each step and what is left to people or external systems.

Step	Prototype	People or external systems
Consent encoding	Compiles DS, with a whitelist of the ICD-10 codes E10–E14, and IS, with an allowlist of requester pseudonyms; commits the policy and registers the resource	Translating the consent into DS, IS, and IRB; deciding which researchers enter the allowlist; handling IRB, which the compiler rejects
Requester evidence	The TPS constructs the witness from the holder secret, the disease code declared in the request, and the allowlist membership path, and generates the proof	Attesting the researcher's affiliation before the provider lists the pseudonym; obtaining and presenting the ethics approval
Authorization decision	The ACM verifies the proof against the policy and parameter commitments and the request context, and records the grant or denial on-chain	Reviewing the ethics approval
Data release	The data access layer checks that the request was granted in the current hour and that the resource is still bound to the active policy	Delivering the data, for example by releasing a decryption key; confirming the ethics approval before release
Consent withdrawal	Revoking the policy, or rebinding the resource to a successor policy, makes earlier grants fail the data-release check	Retracting or deleting data already released
Expiry and change	A changed allowlist or whitelist takes effect through a successor policy; policy validity windows are checked in-circuit	Tracking the expiry of the ethics approval and the end of a researcher's affiliation

DS: disease-specific research; IS: institution-specific restriction; IRB: ethics approval required (Section 2.3).

depend on the vocabulary used at encoding time, when concepts are expanded into fixed code whitelists. Updating these whitelists to reflect later vocabulary revisions requires a successor policy. Deployment therefore requires the provider to review the encoded policy against the original consent and document any conditions that remain outside the proof.

Public blockchain versus an institutional audit log. An institutionally governed access system with an audit log can enforce the same authorization conditions at lower cost and without exposing public-chain metadata. Because the institution maintains the log of its own decisions, auditors rely on it to provide a complete and unaltered record. In TPS-ZKPI, the ACM grants a request only after verifying a proof against the committed policy, and replacing that policy requires a public successor commitment. Any party can therefore verify that each recorded grant is supported by a valid proof for the committed policy, without relying on the operator's account of the decision or joining a consortium.

This verifiability is limited to authorization decisions. The on-chain record does not establish whether data were released only for granted requests or whether ethics approval was reviewed. The permanence of these records must also be reconciled with data-protection requirements such as the right to erasure [51]. Each request additionally incurs gas costs and exposes the public-chain metadata evaluated in Section 6.

A public blockchain is therefore suited to settings that require independently verifiable authorization decisions across institutions without a shared trusted operator. Within a single institution, or where all parties trust a common operator, an institutional access system with an audit log offers a simpler alternative.

Future directions. Based on the TPS trust assumption, its effect on accountability, and the deployment scope of context binding discussed above, we identify several directions for future work. For example, adding an explicit chain/contract domain and an authenticated submission channel would extend context binding beyond a single deployment and reduce front-running risk. In addition, accountable proof obligations, such as on-chain commitments to serve requests combined with timeout-with-evidence mechanisms, would make proof refusals attributable. Finally, the reduced-trust TPS instantiations de-

scribed in Section 4, based on collaborative SNARKs or TEE-backed proving, would relax the trust concentration itself, including the partial delegation of attribute origin.

8. Conclusion

This paper presented TPS-ZKPI, a Trusted Proof Service-assisted zero-knowledge approach to policy-private consent enforcement on public blockchains. In TPS-ZKPI, the data provider commits to a hidden consent policy on-chain, the TPS generates a zero-knowledge proof of each access decision under this policy, and a smart contract verifies the proof. Each policy is compiled into a fixed-length program for a reusable zero-knowledge policy interpreter, so all policies share one verification interface and a fixed-format public-chain transcript. We evaluated TPS-ZKPI on the Polygon Amoy testnet with supervised and unsupervised attacks in which a passive public-chain observer tried to infer hidden policy properties from public-chain metadata. The results showed no leakage under our criterion in any of the 44 supervised tasks, and unsupervised clustering recovered no policy structure beyond chance level. Overall, this paper showed empirically that TPS-ZKPI resists the modeled metadata attacks and provides auditable consent enforcement on a public blockchain.

Author Contributions: Conceptualization, K.L., A.L., M.D. and V.U.; methodology, K.L.; software, K.L.; validation, K.L.; formal analysis, K.L.; investigation, K.L.; data curation, K.L.; writing—original draft, K.L.; writing—review and editing, K.L., A.L., M.D. and V.U.; visualization, K.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was partially funded by the following grants: the China Scholarship Council [202109650003]. The funders had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code of TPS-ZKPI (zero-knowledge circuit, smart contracts, and Trusted Proof Service), the evaluation scripts, the synthetic consent-policy corpus, and the executed dataset supporting the reported results are openly available in Zenodo at https://doi.org/10.5281/zenodo.21335228. The Polygon Amoy transactions in the executed dataset can also be checked on the public testnet.

Acknowledgments: During the preparation of this study, the authors used Claude Code (Anthropic; Claude Opus, Sonnet and Fable models, versions 4.6–5.5), ChatGPT (OpenAI; GPT-5.6 Sol, GPT-6 Sol, GPT-6.1 Sol and GPT-6 Astra models) and the GitHub Copilot coding agent to assist with software development, test and analysis-script writing, and language editing of the manuscript. The authors made all design and methodological decisions, reviewed and edited all AI-generated output, and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ABAC	Attribute-Based Access Control
ABE	Attribute-Based Encryption
ACM	Access-Control Manager
ARI	Adjusted Rand Index
AUC	Area Under the Receiver Operating Characteristic Curve
CI	Confidence Interval
CP-ABE	Ciphertext-Policy Attribute-Based Encryption

DUO	Data Use Ontology
ECDSA	Elliptic Curve Digital Signature Algorithm
EdDSA	Edwards-curve Digital Signature Algorithm
EIP	Ethereum Improvement Proposal
EVM	Ethereum Virtual Machine
GA4GH	Global Alliance for Genomics and Health
HANCESTRO	Human Ancestry Ontology
HE	Homomorphic Encryption
ICD-10	International Classification of Diseases, 10th Revision
IoMT	Internet of Medical Things
LR	Logistic Regression
MONDO	Mondo Disease Ontology
MPC	Multi-Party Computation
NMI	Normalized Mutual Information
OBI	Ontology for Biomedical Investigations
PCH	Policy Content Hiding
PI	Policy Indistinguishability
PRE	Proxy Re-Encryption
RF	Random Forest
RPN	Reverse Polish Notation
SAML	Security Assertion Markup Language
SNARK	Succinct Non-interactive Argument of Knowledge
SNOMED CT	SNOMED Clinical Terms
SPP	Structure and Parameter Privacy
SSI	Self-Sovereign Identity
TEE	Trusted Execution Environment
TPS	Trusted Proof Service
TPS-ZKPI	Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter
WASM	WebAssembly
XACML	eXtensible Access Control Markup Language
zkVM	Zero-Knowledge Virtual Machine

Appendix A. Sample-Size Sensitivity of NMI

The clustering cross-check reads ARI directly but compares NMI against a label-permutation null (Section 6.3). To examine why NMI cannot be read directly, we quantify how *finite-sample effects* alone can inflate NMI even when there is *no* discriminative signal, using an idealized null in which all classes are statistically indistinguishable.

We sample every class from the *same* distribution, $z_i \sim \mathcal{N}(0, I_d)$ for all classes, where I_d is the d-dimensional identity covariance and $d = 2$ matches the two gas features the clustering uses. Since no class carries class-specific information, a clustering algorithm should not recover class membership, and both NMI and ARI should concentrate around 0 as the sample size grows.

For each configuration we run $T = 1000$ independent trials. In each trial we (i) generate G classes with n i.i.d. samples per class from $\mathcal{N}(0, I_d)$, for $n_{\text{tot}} = Gn$ samples in total, assigning each sample a class label by construction order (these labels are independent of the sample values, since all classes share one distribution), (ii) standardize the pooled data and apply K-Means with $K = G$ and ten initializations, as in the evaluation, and (iii) score the predicted assignments against these assigned class labels with NMI and ARI. We vary n while holding G fixed, and report the trial means $\overline{\text{NMI}}$ and $\overline{\text{ARI}}$. We consider a **PI-like** binary structure ($G = 2$, $n \in \{3, 10, 16, 50, 100\}$), a five-class structure ($G = 5$, $n \in \{3, 10, 100\}$), and a ten-class structure ($G = 10$, $n \in \{3, 10, 42\}$) together with the observed ICD-10 chapter sizes (33–55 policies per chapter, one execution each, $n_{\text{tot}} = 422$).

Table A1. Sample-size sensitivity of NMI and ARI under the null hypothesis where all classes are i.i.d. sampled from $\mathcal{N}(0, I_d)$.

n (per class)	n_{tot}	$\overline{\text{NMI}}$	$\overline{\text{ARI}}$
Two classes (as in a PI pair)			
3	6	0.2074	0.0127
10	20	0.0420	0.0008
16	32	0.0250	0.0008
50	100	0.0072	−0.0003
100	200	0.0034	−0.0004
Five classes			
3	15	0.4152	−0.0034
10	50	0.1226	0.0003
100	500	0.0102	−0.0000
Ten classes (as the DUO and ICD-10 targets)			
3	30	0.5346	−0.0009
10	100	0.2087	0.0003
42	420	0.0459	−0.0001
Ten classes, observed ICD-10 chapter sizes			
33–55	422	0.0466	0.0001

$T = 1000$ trials per row, one K-Means fit with ten initializations per trial; n_{tot} is the total sample size. For the observed ICD-10 chapter sizes, the text also reports a separate run that averages each trial over 20 K-Means seeds, as the evaluation does.

Table A1 shows that, although all classes are indistinguishable by construction, **NMI is noticeably positive at very small n but collapses toward zero as n grows**, whereas **ARI stays near zero throughout**. For the binary structure, $\overline{\text{NMI}}$ falls from 0.2074 at $n = 3$ ($n_{\text{tot}} = 6$) to 0.0420 at $n = 10$, 0.0250 at $n = 16$, the size of a designed PI pair (Section 6.5), and 0.0072/0.0034 at $n = 50/100$. The same trend holds with more classes, at a higher level for every n: with five classes $\overline{\text{NMI}}$ goes from 0.4152 ($n = 3$) to 0.0102 ($n = 100$), and with ten classes from 0.5346 ($n = 3$) to 0.0459 ($n = 42$). For the observed ICD-10 chapter sizes, a separate null run that averages each trial over 20 K-Means seeds, as the evaluation does, gives a null NMI with mean 0.0463 and 95th percentile 0.0570. The significance of the observed ICD-10 NMI is assessed by the label-permutation test in Table 11.

References

1. Kakarlapudi, P.V.; Mahmoud, Q.H. A systematic review of blockchain for consent management. *Healthcare* **2021**, *9*, 137. https://doi.org/10.3390/healthcare9020137.
2. Welzel, C.; Ostermann, M.; Smith, H.L.; Minssen, T.; Kirsten, T.; Gilbert, S. Enabling secure and self determined health data sharing and consent management. *npj Digital Medicine* **2025**, *8*, 560. https://doi.org/10.1038/s41746-025-01945-z.
3. Li, K.; Sai, A.R.; Urovi, V. Do you need a blockchain in healthcare data sharing? A tertiary review. *Exploration of Digital Health Technologies* **2024**, *2*, 101–123. https://doi.org/10.37349/edht.2024.00014.
4. Béres, F.; Seres, I.A.; Benczúr, A.A.; Quintyne-Collins, M. Blockchain is watching you: Profiling and deanonymizing Ethereum users. In Proceedings of the 2021 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS). IEEE, 2021, pp. 69–78. https://doi.org/10.1109/DAPPS52256.2021.00013.
5. Li, K.; Lohachab, A.; Dumontier, M.; Urovi, V. Privacy preservation in blockchain-based healthcare data sharing: A systematic review. *Peer-to-Peer Networking and Applications* **2025**, *18*, 1–53. https://doi.org/10.1007/s12083-025-02148-9.
6. Lawson, J.; Cabili, M.N.; Kerry, G.; Boughtwood, T.; Thorogood, A.; Alper, P.; Bowers, S.R.; Boyles, R.R.; Brookes, A.J.; Brush, M.; et al. The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics* **2021**, *1*, 100028. https://doi.org/10.1016/j.xgen.2021.100028.
7. Rondelet, A.; Zajac, M. Zeth: On Integrating Zerocash on Ethereum. arXiv:1904.00905, 2019, [arXiv:cs.CR/1904.00905]. https://doi.org/10.48550/arXiv.1904.00905.

8. Williamson, Z.J. The AZTEC Protocol. https://github.com/AztecProtocol/AZTEC, 2018. Technical whitepaper. Accessed: 2026-09-20.
9. Nadler, M.; Schär, F. Tornado Cash and blockchain privacy: a primer for economists and policymakers. *Federal Reserve Bank of St. Louis Review* **2023**, *105*, 122–136. https://doi.org/10.20955/r.105.122-136.
10. Ibraimi, L.; Petkovic, M.; Nikova, S.; Hartel, P.; Jonker, W. Ciphertext-policy attribute-based threshold decryption with flexible delegation and revocation of user attributes. Technical report, Centre for Telematics and Information Technology, University of Twente, 2009. https://research.utwente.nl/files/5098396/Technical_Report.pdf.
11. Ullah, S.S.; Oleshchuk, V.; Pussewalage, H.S.G. A survey on blockchain envisioned attribute based access control for internet of things: Overview, comparative analysis, and open research challenges. *Computer Networks* **2023**, *235*, 109994. https://doi.org/10.1016/j.comnet.2023.109994.
12. Zhou, L.; Diro, A.; Saini, A.; Kaisar, S.; Hiep, P.C. Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities. *Journal of Information Security and Applications* **2024**, *80*, 103678. https://doi.org/10.1016/j.jisa.2023.103678.
13. Yang, X.; Liu, J. HA-Med: A Blockchain-Based Solution for Sharing Medical Data with Hidden Policies and Attributes. *IET Information Security* **2024**, *2024*, 2498245. https://doi.org/10.1049/2024/2498245.
14. Ding, X.; Liu, Y.; Ning, J.; Chen, D. Blockchain-Enhanced Anonymous Data Sharing Scheme for 6G-Enabled Smart Healthcare With Distributed Key Generation and Policy Hiding. *IEEE Journal of Biomedical and Health Informatics* **2026**, *30*, 666–675. https://doi.org/10.1109/JBHI.2025.3550261.
15. World Health Organization. International Statistical Classification of Diseases and Related Health Problems (10th Revision). https://icd.who.int/browse10/2019/en, 2019. Accessed: 2026-07-08.
16. Dyke, S.O.; Philippakis, A.A.; Rambla De Argila, J.; Paltoo, D.N.; Luetkemeier, E.S.; Knoppers, B.M.; Brookes, A.J.; Spalding, J.D.; Thompson, M.; Roos, M.; et al. Consent codes: upholding standard data use conditions. *PLoS Genetics* **2016**, *12*, e1005772. https://doi.org/10.1371/journal.pgen.1005772.
17. Bethencourt, J.; Sahai, A.; Waters, B. Ciphertext-policy attribute-based encryption. In Proceedings of the 2007 IEEE Symposium on Security and Privacy (SP'07). IEEE, 2007, pp. 321–334. https://doi.org/10.1109/SP.2007.11.
18. Ateniese, G.; Fu, K.; Green, M.; Hohenberger, S. Improved proxy re-encryption schemes with applications to secure distributed storage. *ACM Transactions on Information and System Security (TISSEC)* **2006**, *9*, 1–30. https://doi.org/10.1145/1127345.1127346.
19. Pei, H.; Yang, P.; Li, W.; Du, M.; Hu, Z. Proxy Re-Encryption for Secure Data Sharing with Blockchain in Internet of Medical Things. *Computer Networks* **2024**, *245*, 110373. https://doi.org/10.1016/j.comnet.2024.110373.
20. Zhou, H.; Si, X.M.; Zhao, B.; Zhang, Y.; Qu, Y. Secure electronic medical records sharing scheme based on blockchain by using proxy re-encryption. *Peer-to-Peer Networking and Applications* **2024**, *17*, 4197–4209. https://doi.org/10.1007/s12083-024-01775-y.
21. Groth, J. On the Size of Pairing-Based Non-interactive Arguments. In Proceedings of the Advances in Cryptology – EUROCRYPT 2016. Springer, 2016, pp. 305–326. https://doi.org/10.1007/978-3-662-49896-5_11.
22. Sasson, E.B.; Chiesa, A.; Garman, C.; Green, M.; Miers, I.; Tromer, E.; Virza, M. Zerocash: Decentralized anonymous payments from Bitcoin. In Proceedings of the 2014 IEEE Symposium on Security and Privacy. IEEE, 2014, pp. 459–474. https://doi.org/10.1109/SP.2014.36.
23. Sharma, B.; Halder, R.; Singh, J. Blockchain-based Interoperable Healthcare using Zero-Knowledge Proofs and Proxy Re-Encryption. In Proceedings of the 2020 International Conference on COMmunication Systems & NETworkS (COMSNETS). IEEE, 2020, pp. 1–6. https://doi.org/10.1109/COMSNETS48256.2020.9027413.
24. Madine, M.; Salah, K.; Jayaraman, R.; Yaqoob, I. Zero-knowledge proofs for anonymous authentication of patients on public and private blockchains. *Array* **2025**, *28*, 100590. https://doi.org/10.1016/j.array.2025.100590.
25. Wu, N.; Xu, L.; Zhu, L. A blockchain based access control scheme with hidden policy and attribute. *Future Generation Computer Systems* **2023**, *141*, 186–196. https://doi.org/10.1016/j.future.2022.11.006.
26. Boneh, D.; Sahai, A.; Waters, B. Functional encryption: Definitions and challenges. In Proceedings of the Theory of Cryptography Conference. Springer, 2011, pp. 253–273. https://doi.org/10.1007/978-3-642-19571-6_16.
27. Secret Network. Secret Network Overview. https://docs.scrt.network/secret-network-documentation/overview-ecosystem-and-technology/secret-network-overview. Accessed: 2026-06-29.
28. Cheng, R.; Zhang, F.; Kos, J.; He, W.; Hynes, N.; Johnson, N.; Juels, A.; Miller, A.; Song, D. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In Proceedings of the 2019 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE, 2019, pp. 185–200. https://doi.org/10.1109/EuroSP.2019.00023.
29. Buccafurri, F.; De Angelis, V.; Nardone, R. How can the holder trust the verifier? A CP-ABPRE-based solution to control the access to claims in a Self-Sovereign-Identity scenario. *Blockchain: Research and Applications* **2024**, *5*, 100196. https://doi.org/10.1016/j.bcra.2024.100196.

30. Shashidhara, R.; Chirakarotu Nair, R.; Kumar Panakalapati, P. Promise of Zero-Knowledge Proofs (ZKPs) for Blockchain Privacy and Security: Opportunities, Challenges, and Future Directions. *Security and Privacy* **2024**, *8*, e461. https://doi.org/10.1002/spy2.461.
31. Ardagna, C.A.; De Capitani di Vimercati, S.; Paraboschi, S.; Pedrini, E.; Samarati, P. An XACML-based privacy-centered access control system. In Proceedings of the First ACM Workshop on Information Security Governance, 2009, pp. 49–58. https://doi.org/10.1145/1655168.1655178.
32. Sheikhalishahi, M.; Tillem, G.; Erkin, Z.; Zannone, N. Privacy-preserving multi-party access control. In Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society, 2019, pp. 1–13. https://doi.org/10.1145/3338498.3358643.
33. Gabizon, A.; Williamson, Z.J.; Ciobotaru, O. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019.
34. Campanelli, M.; Fiore, D.; Querol, A. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 2075–2092. https://doi.org/10.1145/3319535.3339820.
35. Liu, T.; Zhang, Z.; Zhang, Y.; Hu, W.; Zhang, Y. Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. *Journal of Cryptology* **2025**, *38*, 17. https://doi.org/10.1007/s00145-024-09533-2.
36. Miller, T.; Alvarez, B.; Hoang, T. VTBC: Privatizing the Volume and Timing of Transactions for Blockchain Applications. In Proceedings of the 2023 32nd International Conference on Computer Communications and Networks (ICCCN). IEEE, 2023, pp. 1–10. https://doi.org/10.1109/ICCCN58024.2023.10230098.
37. Hu, R.; Ma, Z.; Li, L.; Zuo, P.; Li, X.; Wei, J.; Liu, S. An access control scheme based on blockchain and ciphertext policy-attribute based encryption. *Sensors* **2023**, *23*, 8038. https://doi.org/10.3390/s23198038.
38. Jaiman, V.; Urovi, V. A Consent Model for Blockchain-Based Health Data Sharing Platforms. *IEEE Access* **2020**, *8*, 143734–143745. https://doi.org/10.1109/ACCESS.2020.3014565.
39. Bernstein, D.J.; Duif, N.; Lange, T.; Schwabe, P.; Yang, B.Y. High-speed high-security signatures. *Journal of Cryptographic Engineering* **2012**, *2*, 77–89. https://doi.org/10.1007/s13389-012-0027-1.
40. WhiteHat, B.; Bellés, M.; Baylina, J. EIP-2494: Baby Jubjub Elliptic Curve. https://eips.ethereum.org/EIPS/eip-2494, 2020. Accessed: 2026-09-24.
41. Grassi, L.; Khovratovich, D.; Rechberger, C.; Roy, A.; Schofnegger, M. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In Proceedings of the 30th USENIX Security Symposium, 2021, pp. 519–535.
42. Barreto, P.S.L.M.; Naehrig, M. Pairing-Friendly Elliptic Curves of Prime Order. In Proceedings of the Selected Areas in Cryptography (SAC 2005). Springer, 2006, Vol. 3897, *Lecture Notes in Computer Science*, pp. 319–331. https://doi.org/10.1007/11693383_22.
43. Reitwiessner, C. EIP-196: Precompiled Contracts for Addition and Scalar Multiplication on the Elliptic Curve alt_bn128. https://eips.ethereum.org/EIPS/eip-196, 2017. Accessed: 2026-10-05.
44. Akhunov, A.; Ben Sasson, E.; Brand, T.; Guthmann, L.; Levy, A. EIP-2028: Transaction Data Gas Cost Reduction. https://eips.ethereum.org/EIPS/eip-2028, 2019. Accessed: 2026-10-05.
45. Bandrowski, A.; Brinkman, R.; Brochhausen, M.; Brush, M.H.; Bug, B.; Chibucos, M.C.; Clancy, K.; Courtot, M.; Derom, D.; Dumontier, M.; et al. The Ontology for Biomedical Investigations. *PLoS ONE* **2016**, *11*, e0154556. https://doi.org/10.1371/journal.pone.0154556.
46. Hubert, L.; Arabie, P. Comparing partitions. *Journal of Classification* **1985**, *2*, 193–218. https://doi.org/10.1007/BF01908075.
47. Vinh, N.X.; Epps, J.; Bailey, J. Information Theoretic Measures for Clusterings Comparison: Variants, Properties, Normalization and Correction for Chance. *Journal of Machine Learning Research* **2010**, *11*, 2837–2854.
48. Cox, D.R. The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society: Series B (Methodological)* **1958**, *20*, 215–232. https://doi.org/10.1111/j.2517-6161.1958.tb00292.x.
49. Breiman, L. Random Forests. *Machine Learning* **2001**, *45*, 5–32. https://doi.org/10.1023/A:1010933404324.
50. Lloyd, S. Least squares quantization in PCM. *IEEE Transactions on Information Theory* **1982**, *28*, 129–137. https://doi.org/10.1109/TIT.1982.1056489.
51. Haque, A.B.; Islam, A.N.; Hyrynsalmi, S.; Naqvi, B.; Smolander, K. GDPR compliant blockchains–a systematic literature review. *IEEE Access* **2021**, *9*, 50593–50606.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.